COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

C++, Algorithms and Language Implementation

Student handbook / expanded Volume 03

20 lessons · Modules 11–15 · Worked examples · Practical work · 100 lesson questions

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

C++, Algorithms and Language Implementation 3

Module 11 — Native programming with C++ 4

Module 12 — Ownership, abstractions and C++ design 17

Module 13 — Data structures and their costs 30

Module 14 — Algorithms for maps, simulation and tools 42

Module 15 — Programming languages, compilers and computability 54

Sources and provenance 67

C++, Algorithms and Language Implementation

How to use the expanded volume

Read START_NEXT.pdf before running code. This handbook preserves lessons11.1–15.4 and expands their explanations, worked examples and practical work. Each lesson ends with five questions; the first three are original and the final two are new. The separate workbook supplies additional exercise depth and integration projects. Code under reference is complete teaching material; code under starter is intentionally incomplete student work.

The goal is to explain and test what a native program does, not merely make an engine editor display a scene. Follow the written domains and lifetime rules. Predict a result, implement a small change, then check it. Finish the assigned primary study and preserve the evidence described in each lab. GateC integrates this work; it is not already completed by the reference programs.

Module 11 — Native programming with C++

Prerequisites: Volume 01, Modules 06-07; do not skip the ownership module that follows.

Project: Port the deterministic grid simulation to C++20, with a command-line build, automated tests, warnings enabled and an explicit account of representation differences from Python.

Primary references: R11, R12, R15. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

11.1 — Compilation, linking and build systems

Learning outcome: Explain how source code becomes a runnable native program.

From a Python script to a native program

A source file is not an executable. In this bundle, CMake describes targets and dependencies; a generator produces build instructions; a compiler translates each C++ translation unit; a linker combines the resulting object files and required libraries. The operating system then loads the executable. A successful compile is evidence about one stage, not proof that the whole program is correct. A game can build, launch and still update the wrong entity.

A translation unit is approximately a source file after preprocessing, including the text brought in through headers. A header normally exposes declarations that several translation units need. The definition of an ordinary non-inline function belongs in one source file. Include guards prevent duplicate inclusion within a translation unit; they do not magically permit multiple ordinary function definitions across different translation units. Later you will meet the permitted multiple definitions of inline functions and templates.

Keep four questions separate: Can the compiler find the declaration? Does a matching definition exist? Did the linker receive the object or library containing it? Does the program satisfy the runtime contract? This separation turns a screen of diagnostics into a sequence of testable hypotheses. [R11: SF; R12]

Reading the smallest useful C++ program

The included example declares `int next_tick(int current);`, defines it in another file, and calls it from `main`. The semicolon terminates a declaration or statement. Braces define a body or scope. The return type precedes the function name; argument types are explicit. `#include` is a preprocessing directive, not a runtime function call. `std::cout` names the output object in the standard namespace; << streams a value into it.

Use a function's declaration as a contract, not merely a compiler appeasement. Our next-tick function accepts an integer below the maximum representable value and rejects the upper boundary before adding. A function named `next_tick` that silently wraps time would introduce an observable simulation rule, not a harmless implementation detail. Tests express that choice.

The `main` function returns zero on successful completion. Our test harness returns nonzero when a check fails, and CTest uses that status. Do not use an ordinary C `assert` as the only test mechanism: optimized builds may define `NDEBUG` and remove it. The included CHECK helper remains active in both Debug and Release configurations.

Targets, configuration and reproducibility

A CMake target gathers source files, include directories, language requirements and link dependencies under one name. Prefer target-scoped settings to machine-wide flags. The bundle requires C++20 explicitly and disables compiler extensions for the teaching examples. That is a selected baseline, not a claim about the newest language or complete compiler feature support.

An out-of-source build separates the authored files under `code/` from generated products under `build/`. A clean build can be reproduced without relying on yesterday's object files. Configure once, build the selected targets, then run tests. Reconfigure when toolchain or build options change. Never compare timings from an instrumented Debug build with an optimized uninstrumented Release build and attribute the entire difference to an algorithm.

Record the compiler identity, compiler version, standard-library implementation when known, CMake version, configuration and command line. Multi-configuration generators select Debug or Release at build time; single-configuration generators commonly select through `CMAKE_BUILD_TYPE`. The getting-started guide supplies commands for both. [R12; P4-01]

Diagnose one stage at a time

A missing semicolon is ordinarily a compile-time syntax error. A call with no visible declaration is also a compilation problem. A visible declaration with an omitted implementation can compile but fail to link. Two conflicting definitions may produce a duplicate-symbol failure. A wrong signature in the implementation can accidentally define a different overload, leaving the intended one unresolved.

Read the first relevant diagnostic before fixing the later cascade. Reduce the failing case to one target and preserve the exact command and message in the lab journal. Do not cure a link error by including a `.cpp` file inside another `.cpp` file; add that source to the target. Do not put `using namespace std;` in a public header, because it imposes names on every includer.

The broken examples are intentionally stored as text fixtures outside the default build. The optional diagnostic script runs a syntax-failure and an unresolved-symbol experiment and checks that both are rejected. There is no requirement to execute undefined behavior to learn what an invalid program looks like.

Trace a two-file build

`tick.hpp` declares `next_tick`. `tick.cpp` includes that header and supplies the body. `lesson_11_1.cpp` includes the same header and calls the function. Compiling the main file alone knows the argument and result types, but its object file can still contain an unresolved reference. Linking both object files provides the implementation.

Predict three experiments before running them: remove the declaration from the header; restore it and omit `tick.cpp` from the link; restore both files. The expected outcomes are compilation failure, link failure and successful execution. Record the actual diagnostic wording on your compiler rather than memorizing one vendor's message.

For input 41 the result is 42. At the maximum integer the function throws before arithmetic. The test catches that error. A clean build therefore demonstrates both the translation path and one explicit domain boundary.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_11_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// tick.hpp
#pragma once
int next_tick(int current);

// tick.cpp
#include "tick.hpp"
#include <limits>
#include <stdexcept>
int next_tick(int current) {
    if (current == std::numeric_limits<int>::max())
        throw std::overflow_error("tick limit");
    return current + 1;
}
```

Common failure modes

A header that happens to compile because another header included a dependency is fragile: include what the header itself uses. A stale executable can appear to run after a failed build; verify the build exit status. Compiler warnings are evidence to investigate, not clutter to disable globally.

Practical assignment

Original requirement: Build the included C++ starter, then split one function into a header and source file. Pass: reproduce and explain one compile error and one link error, then restore a warning-clean build.

1. Configure and build the reference target. Save the full toolchain fingerprint and identify each build stage.
2. Recreate the two-file target in the starter directory. Change the function name consistently, then introduce and repair one declaration/definition mismatch.
3. Use the diagnostic fixtures to distinguish a compile error from a link error. Keep both messages in the journal.
4. Add a boundary test and rebuild from a new build directory. Explain why the test remains active in Release.

Required cases: ordinary input 0 -> 1; ordinary input 41 -> 42; maximum int -> overflow error before addition; missing declaration -> compile rejection; missing definition -> link rejection.

Evidence to submit: Two source files and a header, reproducible commands, actual diagnostic transcripts, a warning-clean restored build, and a labeled compile/link/load diagram.

Independent transfer: A teammate supplies a declaration taking `long` but a definition taking `int`. Explain why the implementation may not satisfy the call, then repair the mismatch without a cast at the call site.

Assigned primary reading: R12: Getting Started and libraries; R11: Source Files.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What task does the linker perform?
2. Can valid declarations alone supply every function implementation?
3. Why record build configuration with a benchmark?
4. A build reports failure, but an executable still exists. Is its output evidence that your edit works?
5. Why does an include guard not solve ordinary duplicate function definitions in two translation units?

11.2 — Integer, floating-point and object representations

Learning outcome: Choose native types with explicit range and precision assumptions.

Values are mathematical; representations are finite

A game rule might describe an unbounded score, but a native integer represents a finite set. Eight bits provide 2^8 = 256 patterns. The meaning assigned to those patterns depends on the type. `std::numeric_limits<T>` describes limits on the current implementation; `sizeof(T)` counts C++ bytes, and `CHAR_BIT` states the number of bits in one byte. Do not silently assume every target has the same `int`, pointer or alignment.

Use `std::size_t` for object sizes and container indices where the library requires it, but do not use unsigned arithmetic as a substitute for domain validation. Converting a negative index to an unsigned value does not make it valid. Exact-width types such as `std::uint32_t` are useful when the implementation provides them; a file format must still specify byte order and an encoding independent of native object layout.

The first experiment prints widths and limits on your toolchain. Write down an assumption before printing the answer. A mismatch is useful evidence, not a failed lesson. The representation report belongs in the simulation port's handoff. [R11: ES; R15]

Reject invalid arithmetic before performing it

For unsigned addition modulo 2^N, wraparound is defined by the language, but it may still be wrong for the application. Ordinary signed overflow is undefined behavior in C++. A check after `a + b` is too late if that sum overflowed. To add signed integers safely, check `a > max - b` when `b > 0`, and `a < min - b` when `b < 0`; only then perform the addition.

For a buffer of `count` elements of size `element_size`, first require `element_size != 0` where that is part of the contract. To cap allocation by `limit`, reject `count > limit / element_size`; then multiplication cannot exceed the chosen limit. The limit might be lower than the machine maximum because the engine has a resource budget.

Parsing and converting are separate steps. Parse a nonnegative decimal quantity into a sufficiently wide unsigned type with a checked parser, verify the entire input was consumed, apply the application limit and then convert. Inputs such as `-1`, `12junk`, empty text and an excessive number must not turn into successful lengths.

Floating-point behavior needs a contract

A floating-point number stores a sign, a scale and a finite-precision significand in common binary implementations. Many decimal fractions have no finite binary expansion, so an operation rounds. Precision is not the same as range: a type can represent enormous magnitudes while losing the ability to distinguish nearby small differences at those magnitudes.

Use exact equality when testing an exact discrete decision or a deliberately exact arithmetic case. For approximate numerical outputs, choose an application-specific combined check: absolute error <= absolute tolerance + relative tolerance times the larger magnitude. This is a test policy, not a universal equivalence relation suitable for sorting. NaN is unordered in ordinary comparisons; explicitly reject nonfinite values where the simulation contract requires finite numbers.

Do not promise bit-identical floating-point replay across arbitrary compilers, architectures and optimization flags. This volume's simple grid uses bounded integers to make the state rules easier to reproduce. Later physics work needs its own deterministic or tolerance-based comparison policy. [R15; P4-02]

An object in memory is not a file format

A structure may contain padding between fields and at its end. Padding, byte order, field width and alignment can differ between targets. Saving `sizeof(struct)` bytes also exposes implementation details that become incompatible when the structure changes. Pointer values have no useful portable meaning in a save file.

Specify a wire or disk record field by field. For this lesson, encode a 32-bit unsigned value as four octets in little-endian order. The low byte contains bits 0 through 7; the next contains bits 8 through 15, and so on. The receiving function reconstructs the value with shifts on an appropriately wide unsigned type. This is independent of the host's byte order.

Byte inspection is an observation exercise, not permission to reinterpret unrelated object types arbitrarily. Use facilities intended for object representation and respect their constraints. Our teaching encoder deals with values, not uninitialized padding. The later C bridge prints actual structure offsets and then deliberately avoids using that raw layout as the packet format.

A checked asset allocation

Suppose an asset permits at most 1,024 bytes and each record needs 24 bytes. Testing `count > 1024 / 24` rejects count 43, because integer division yields 42. Count 42 produces 1,008 bytes and is accepted. No potentially overflowing multiplication is needed to discover that 43 is invalid.

For a 32-bit record value `0x12345678`, the defined little-endian bytes are `78 56 34 12` in hexadecimal. Decode by OR-ing the four unsigned byte values shifted by 0, 8, 16 and 24 bits. The round trip checks the encoding, but a second test must compare the encoded bytes with the independently specified sequence: a mutually wrong encoder and decoder can still round-trip.

For an external length, check negative syntax, the parser's out-of-range result, trailing characters and the application budget. A cast belongs after these decisions, not before them.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_11_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
bool fits_budget(std::size_t count,
                 std::size_t element_size,
                 std::size_t byte_limit) {
    return element_size != 0 &&
```

```
            count <= byte_limit / element_size;
}
```

Common failure modes

Never make signed overflow occur and then inspect whether it wrapped. Do not use an arbitrary epsilon for every number. A successful native-layout round trip on one machine does not establish portable serialization.

Practical assignment

Original requirement: Print relevant sizes and numeric limits on the available compiler. Write checked conversions for a file length and tests for boundary values. Pass: do not trigger signed overflow as a test oracle; reject out-of-range input before converting.

1. Print numeric limits, sizes, byte width and selected alignment values. Mark which are observations rather than portable constants.
2. Implement checked decimal length parsing and pre-multiplication budget checks.
3. Write a field-based little-endian encoder/decoder and compare with independently written bytes.
4. Test numeric boundaries and document the difference between Python integers and the chosen native representation.

Required cases: 0 and exact application maximum accepted; negative text and trailing junk rejected; one above the application limit rejected; huge decimal value rejected by parser; 42 records of 24 bytes accepted under 1024; 43 rejected; 0x12345678 encodes to 78 56 34 12.

Evidence to submit: Representation report, boundary-test table, checked conversion implementation, specified encoding and a comparison with the earlier Python model.

Independent transfer: A texture claims width = 100,000 and height = 100,000. Design checks for width * height * bytes_per_pixel without overflowing during validation.

Assigned primary reading: R11: arithmetic and conversion guidance; R15: information encoding and arithmetic.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. How many distinct bit patterns fit in eight bits?
2. Is ordinary signed integer overflow defined to wrap in C++?
3. Why is writing raw struct bytes not a portable format?
4. Why is `if (a + b > max)` not a safe signed-overflow check?
5. Can a serializer and deserializer round-trip correctly yet disagree with the file specification?

11.3 — Values, references, pointers and lifetimes

Learning outcome: Track where an object lives and how long access remains valid.

Identity, copying and borrowing

A value parameter creates a separate parameter object. For a small position or entity identifier, copying often expresses the intended independence. A reference names an existing object; a pointer stores an address-like value that can be null when the interface permits it. Neither a raw pointer nor a reference communicates ownership automatically.

`const T&` lets a function borrow an existing object without copying and prevents mutation through that reference. It does not keep the object alive. It does not prevent another alias from changing the object. It also does not turn a type with hidden mutable state into a thread-safe object. A useful interface states not only what may be modified, but for how long access is retained.

Our damage calculation copies two small integers. Our read-only unit lookup borrows a container. A nullable pointer parameter has an explicit policy: null means no target and returns a not-found result, not an unchecked dereference. Choose parameter form from these semantics rather than from a blanket rule that pointers are fast or references are safe. [R11: F, I, R]

Storage duration is not the entire lifetime story

An automatic local object normally lives until its scope exits. A dynamically allocated object lives until its owner releases it. A container owns its elements, but a reallocation can end the old element lifetimes even while the container itself remains alive. Therefore a pointer's numerical address is not a sufficient proof that it designates the object you intended.

A function must not return a reference or pointer to a local variable that is about to be destroyed. Returning a value is often the simplest repair. Returning a view into caller-owned data can be valid when the caller retains the owner and performs no invalidating operation. Document that dependency explicitly.

Some C++ rules extend certain temporary lifetimes, but not every chain of references extends a temporary. For this course, avoid storing a borrowed reference to a temporary. A `std::string_view` or `std::span` owns no storage. Constructing a view from a short-lived owner and keeping the view is a common lifetime defect, even though the view object itself looks ordinary.

Draw an ownership and access diagram

For each important object, label one owner, zero or more borrowers, and the operations that end or relocate it. Draw arrows from borrowers to the owner. A unit vector owns its unit values. A lookup result can borrow one unit until the next invalidating mutation; an integer unit ID is a value that can be looked up again after mutation.

Stable identity is a separate design problem from stable address. An index remains an integer after vector growth, but erasing or reordering elements may make it refer to a different entity. A later engine can use generation-counted handles to detect recycled slots. Here, keep IDs unique and perform explicit lookup instead of pretending an index is a permanent entity identity.

The lab asks you to analyze a dangling-reference example as text. Do not execute the invalid access to see whether it crashes. Undefined behavior need not produce a crash and is not a reliable teaching oracle. Sanitizers add useful checks on paths that actually run, but an unreported defect is not a proof of lifetime safety. [P4-03; P4-04]

Bounds belong with the borrow

Passing a raw pointer to the first element without a length loses the range boundary. `std::span<const Unit>` carries a contiguous range and its size while remaining non-owning. The caller must still keep the elements alive. A span is not a memory-ownership system and ordinary indexing is not an automatic guarantee of checked bounds in the C++20 baseline.

When the function should consume only a subset, pass a bounded range or explicit start/end convention. Validate external indices before accessing elements. Keep the borrowed reference local to the operation rather than storing it in long-lived engine state. A clear lifetime usually does more for reliability than a clever pointer trick.

In the code example, a query returns an optional copied position rather than a reference. That choice spends a small copy to avoid exporting a lifetime obligation. For large assets, a borrowed view can be appropriate, but the interface must state the mutation and destruction events that invalidate it.

A lookup that does not export a dangling reference

A vector contains unit 7 at position (2,3). `position_of` scans a borrowed span and returns an optional Position value. Finding unit 7 copies (2,3) into the result. The caller can grow or destroy the vector without invalidating that copied result. A missing ID returns no value, not a fabricated position (0,0).

Compare a function returning `const Position&` from a local `Position p`. The local lifetime ends at return, so the reference is invalid immediately. The correct repair is not to make the reference `const`; it is to return a value or choose an owner whose lifetime actually covers use.

Draw the diagram twice: once for a copied result and once for a borrowed pointer into the vector. List the new obligations introduced by the borrowed version.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_11_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
std::optional<Position> position_of(
    std::span<const Unit> units, int id) {
    for (const Unit& u : units)
        if (u.id == id) return u.position;
    return std::nullopt;
}
```

Common failure modes

`const` is not a lifetime extension. A non-null pointer can still dangle. An index can remain in bounds yet name the wrong entity after an erase. A view copied by value still borrows its underlying storage.

Practical assignment

Original requirement: Implement functions taking values, const references and pointers with a documented null policy. Pass: explain each lifetime and identify a deliberately supplied dangling-reference example without executing undefined behavior.

1. Implement equivalent small functions taking a value, a const reference and a pointer with an explicit null policy.
2. Draw owners and borrowers for each function call. State what happens after a vector growth or owner destruction.
3. Implement a copied-value lookup and contrast it with a short-lived borrowed lookup.
4. Diagnose the supplied dangling example without executing it; submit the corrected version and reasoning.

Required cases: empty range -> absent; present ID -> copied expected position; missing ID -> absent; null optional target -> explicit result; copied result remains valid after source growth; dangling local identified by inspection.

Evidence to submit: Parameter-choice table, lifetime diagrams, null-policy tests and a repaired invalid-reference example.

Independent transfer: An event callback captures a local variable by reference and runs next frame. Show two safe designs and explain which one owns the captured state.

Assigned primary reading: R11: Interfaces, Resource Management and lifetime safety guidance.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does a raw pointer necessarily own its target?
2. Why is returning a reference to a local variable invalid?
3. Does const by itself make shared access thread-safe?
4. A `std::span` is copied into a job that runs after its vector is destroyed. Is the copy safe?
5. Why might returning a small position by value be better than returning a reference?

11.4 — Standard containers, iterators and error handling

Learning outcome: Use library abstractions while respecting invalidation and failure contracts.

Use the library, but learn the contract

A standard container packages allocation and element management behind a tested interface. It does not remove the need to understand its invariants. For `std::vector`, size is the number of constructed elements; capacity is the storage available before a growth allocation is needed. `reserve(100)` can increase capacity without creating 100 elements. `resize(100)` changes the element count.

A vector is a good default when the work needs contiguous storage, indexed access and frequent iteration. The teaching goal is not to replace the standard library with custom engine containers as early as possible. Implement educational containers later to understand costs, then justify any production replacement with measured requirements and a maintenance budget.

A range-based loop can read each element by const reference. A loop that copies each large object has different costs and possibly different behavior. Iterator categories also matter: an algorithm requiring random access cannot be assumed to work equally on a linked structure. State what operations the algorithm actually needs. [R11: SL; R13]

Invalidation is part of the interface

Vector reallocation invalidates pointers, references and iterators to its elements. Appending without reallocation preserves existing element references, but the past-the-end iterator changes. Erasing elements invalidates iterators and references at or after the erased position, even if no allocation occurs. Do not keep an iterator across a mutation just because capacity did not change.

Use the iterator returned by `erase` when removing elements during traversal, or use an appropriate erase algorithm. When entity references must survive structural changes, store an ID and re-query or design a stable-handle scheme. Reserving capacity is a useful allocation-control tool, not a general lifetime cure.

The lab deliberately forces growth and erasure, but it never dereferences a known-invalid pointer. Instead it validates results through fresh lookup and describes the invalidation event in the journal. This separates observing allocation behavior from performing an invalid memory access.

Expected absence versus an exceptional failure

A lookup that does not find a unit is often normal control flow. An optional result expresses this without a magic integer or a thrown exception. A malformed asset, an allocation failure and a violated internal invariant are different categories and may deserve different responses. Define the error taxonomy before scattering catches throughout the engine.

This bundle uses `std::optional` for expected absence and exceptions for selected failure paths. C++20 does not provide `std::expected` as a standard facility, so the examples do not depend on it. A production no-exceptions subsystem can use a result type, but must still preserve the same ownership and failure-state reasoning. Error transport does not replace an error contract.

Do not catch everything and continue as though loading succeeded. A catch block should recover, translate a failure at an appropriate boundary, add context, or terminate the operation cleanly. RAII in the next module keeps resources correct as an exception unwinds through the code. [R11: E]

Make mutation explicit and testable

The unit repository accepts unique IDs, appends units, finds by ID and removes by ID. A duplicate insertion returns a clear failure. A lookup returns a copied Unit in the teaching version, so external callers cannot keep stale element references. That choice also prevents callers from changing IDs behind the repository's invariant.

Separate a successful absence from a broken invariant. Removing a missing ID can return false without changing the container. Inserting an already-present ID should not silently create ambiguous lookup behavior. A test after every mutation verifies unique IDs and expected size.

This small API creates a path to the later resource manager: hide implementation details, publish ownership and error behavior, and test boundaries. The container implementation is important, but the observable semantics are what the game depends on.

Trace growth and removal without stale access

Start with an empty vector, reserve space, insert IDs 4 and 9, and query ID 4. The optional result is a value copy. Append enough distinct units to exceed the recorded capacity; do not assume a particular growth factor. Query ID 4 again and compare its value. Remove ID 4, then query it once more and expect absence.

For erase-during-iteration, trace a vector with hit points [0,5,0]. Erasing the first element shifts 5 and 0 toward the beginning. Incrementing an old iterator unconditionally can skip an element or use an invalid iterator. Assign the iterator returned by erase instead.

Neither `reserve` nor a check that an address looks unchanged grants permission to retain invalidated iterators.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_11_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
for (auto it = units.begin(); it != units.end();) {
    if (it->hit_points == 0)
        it = units.erase(it);
    else
        ++it;
}
```

Common failure modes

Capacity is not size. A vector erase can invalidate references even without growth. A sentinel such as ID -1 can collide with the domain unless the domain excludes it. An empty catch block can turn a visible failure into corrupted game state.

Practical assignment

Original requirement: Port the unit list to std::vector and implement lookup with an explicit not-found result. Pass: test empty input, missing ID and growth; document which operations invalidate stored references.

1. Port the unit list to a vector-backed API with unique IDs and copied optional lookup results.
2. Test empty, present and missing queries, duplicate insertion, growth and erasure.
3. Write an invalidation table for reserve, append, erase and clear. Include the past-the-end iterator.
4. Classify normal absence, bad external input and invariant failure; explain the chosen error channel for each.

Required cases: empty lookup -> absent; duplicate ID -> rejected with unchanged size; growth preserves logical ID lookup; missing removal -> false and unchanged state; erase adjacent zero-HP units correctly; reserve changes capacity, not constructed count.

Evidence to submit: Unit repository, automated tests, error taxonomy and operation-specific invalidation table.

Independent transfer: A renderer stores vector iterators across an update that removes dead units. Redesign the frame boundary so rendering never uses invalid iterators.

Assigned primary reading: R11: standard-library use and error handling; R12: testing integration.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does reserve(100) make a vector's size 100?
2. Why can a pointer to a vector element become invalid after growth?
3. What is wrong with ignoring every exception?
4. Can vector erase invalidate a reference when capacity stays constant?
5. When is an optional copied value a suitable lookup result?

Module 12 — Ownership, abstractions and C++ design

Prerequisites: Module 11.

Project: Build a small resource manager with explicit ownership, safe move behavior, bounded interfaces, fault-injection tests and a written comparison of composition and inheritance.

Primary references: R11, R12. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

12.1 — RAII and resource ownership

Learning outcome: Make cleanup follow object lifetime on success and failure paths.

A resource is more than heap memory

Files, device handles, locks and temporary registrations all have acquisition and release rules. Resource Acquisition Is Initialization connects successful acquisition to an object's lifetime. A constructed owner is responsible for release when it leaves scope, including ordinary early returns and exception unwinding. The destructor provides the cleanup path rather than requiring every caller to remember it.

The resource-owning object must know whether it actually acquired the resource. A default-constructed or moved-from owner may be empty. Its destructor must not close an invalid handle. If construction fails after an owning member is fully constructed, that member is destroyed, even though the enclosing object's destructor is not run for the incomplete object.

Prefer established owning members such as `std::unique_ptr` and standard containers. The custom mock owner in this lesson is educational: it exposes acquisition/release counts so you can verify the invariant. Real graphics-device wrappers add device affinity, synchronization and destruction-order constraints beyond this mock. [R11: R, E]

One owner is usually easier to reason about

Unique ownership means one object controls the resource's release. A borrowed pointer can refer to the resource temporarily without participating in ownership. Shared ownership has a different contract: the resource survives until the last owning reference is released. That can be useful, but reference counting and delayed destruction introduce costs and timing obligations.

Owning cycles can prevent shared objects from reaching a reference count of zero. Break a relationship that is semantically observational with a weak or non-owning link; do not make every edge owning by default. More fundamentally, ask which subsystem should govern the resource's lifetime.

A resource manager often owns assets while users hold stable IDs or scoped borrows. The choice depends on whether an asset may be unloaded while a job still uses it. This volume stays single-threaded, but the ownership diagram should already expose where concurrency would need additional coordination.

Failure paths are first-class behavior

Imagine opening a file, then allocating a buffer, then parsing metadata. Parsing can fail after both acquisitions. Manual code needs a correct release on every exit, including future exits added by maintenance. RAII owners make the cleanup order follow scope instead.

The lab tracks acquired, released and currently live resources. After a normal operation, live count returns to the baseline. After an injected failure immediately after acquisition, it also returns to baseline. The important assertion is not merely that the program did not crash; it is that every successful acquisition was released exactly once.

Cleanup must not throw from an ordinary destructor used during stack unwinding. If a release operation can report an important failure, provide an explicit close/flush operation that returns or throws in a controlled context, and make destructor cleanup a documented fallback. A mock counter cannot establish the behavior of a real device API.

Respect destruction dependencies

C++ members are initialized in their declaration order and destroyed in reverse declaration order, regardless of the order written in the constructor's initializer list. If an asset owner needs the device alive during release, declare and construct the device before the dependent asset owners, or use a higher-level scope that enforces the same dependency.

A borrowed view must not outlive its resource owner. RAII solves release ownership, not all dangling borrows. An object's destructor can be perfectly correct while another subsystem still keeps a pointer to the destroyed resource. That is why the earlier ownership diagram remains necessary.

In a game shutdown sequence, stop new work, finish or cancel outstanding uses, release dependent resources, and finally release the underlying device. Later systems modules will add job synchronization and GPU completion. Here you must articulate the order without pretending the mock device has those real-world guarantees.

Inject a failure after acquisition

Begin with live = 0, acquired = 0 and released = 0. Construct the mock owner: live = 1, acquired = 1. Trigger a controlled exception before normal completion. Stack unwinding destroys the owner: live = 0, released = 1. The outer catch records the failure. Repeat with a successful path and verify the counts advance to acquired = released = 2.

Now create an empty owner. Its destruction must leave the counters unchanged. These three paths distinguish successful acquisition, failed work after acquisition and no acquisition at all.

For an enclosing object whose second member fails to construct, trace which earlier member owners already exist. Their destructors still run; the incomplete outer object does not get its normal destructor call.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_12_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
void load_with_failure(bool fail_after_acquire) {
    MockOwner resource;
    if (fail_after_acquire)
        throw std::runtime_error("injected load failure");
    // resource is released at scope exit on either path
}
```

Common failure modes

RAII does not make a borrowed pointer immortal. Shared ownership is not a universal leak cure. Constructor initializer-list order does not override member declaration order. A destructor that throws during unwinding can terminate the process.

Practical assignment

Original requirement: Wrap a mock file/device resource whose open/close counters can be inspected. Inject failure after acquisition. Pass: each acquired resource is released exactly once and no view outlives its owner.

1. Implement or inspect a mock resource with observable acquisition and release counters.
2. Wrap it in a non-copying RAII owner and test normal completion, early return and injected failure.
3. Add an enclosing object with dependent members and explain partial-construction cleanup.
4. Draw the device-before-resource construction and resource-before-device destruction order.

Required cases: successful scope releases exactly once; injected exception releases exactly once; empty owner releases nothing; nested owners unwind in reverse order; all paths return live count to baseline.

Evidence to submit: Acquisition/release ledger, fault-injection tests, ownership diagram and destruction-order explanation.

Independent transfer: A file close operation can fail when flushing buffered data. Separate explicit error-reporting close from destructor cleanup without allowing two releases.

Assigned primary reading: R11: Resource Management and smart-pointer ownership rules.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What does RAII connect?
2. Can shared ownership create a leak through cycles?
3. Why must device-resource destruction order be planned?

4. Does an enclosing object whose constructor throws get its normal destructor call?
5. Why can correct RAII still coexist with a use-after-free bug?

12.2 — Copy, move and exception safety

Learning outcome: Define what transferring a resource means without duplicating ownership.

Copying a value is not duplicating an owner

A value such as a position can be copied naturally. A device handle often represents a unique obligation to release one resource. Copying its integer handle into another independent owner would give two destructors the same obligation. Delete copy operations for a unique owner unless the resource API supplies a real duplication operation with clear semantics.

A move transfers ownership into a destination and leaves the source in a valid state. For the educational handle, the source becomes empty. Standard-library moved-from objects commonly have a valid but unspecified state unless their contract says more; do not generalize the mock's empty-state guarantee to every type.

`std::move` is a cast that allows move-aware overload selection. It does not itself transfer bytes, release anything or guarantee that a move overload exists. Moving a const object can select a copy because ordinary moves need to modify the source. Read the selected operation rather than treating the word move as a magical effect. [R11: C.20, C.21, C.64]

Prefer the rule of zero; learn special members deliberately

The rule of zero means composing a class from members that already implement the desired ownership semantics so custom copy, move and destruction are unnecessary. A vector of values and a unique pointer to a mock resource can manage themselves. Writing five special member functions for every class increases the number of invariants you must maintain.

When a custom handle wrapper is justified, make its contract explicit: default construction is empty; copying is prohibited; move construction steals ownership; move assignment releases the old destination resource before taking the source; destruction releases a nonempty handle. Guard self-move assignment or define an implementation that remains valid under it.

Mark a move `noexcept` only when it really cannot throw. Containers can use this information when deciding how to preserve their own guarantees during relocation. A false `noexcept` promise is dangerous: throwing through it terminates instead of allowing ordinary recovery.

Three useful levels of failure guarantees

The basic exception guarantee preserves valid invariants and avoids leaks, but values may change. The strong guarantee leaves relevant observable state unchanged on failure. A no-throw guarantee promises the operation does not propagate an exception. These are interface commitments, not optimistic comments.

For asset replacement, load and validate a candidate without touching the current asset. Only after all fallible preparation succeeds should a nonthrowing commit swap the candidate into the current slot. If loading fails, the old asset remains accessible. After successful commit, the old asset is released through the temporary candidate's destruction.

Define what observable state means. Logging a failed attempt or incrementing a diagnostic counter may be permitted even while the asset binding remains unchanged. State the boundary so tests do not confuse the asset guarantee with a promise that absolutely no bit anywhere changes.

Failure injection tests the design, not luck

A successful run tests only the successful path. Add controlled failures after acquisition, during validation and before commit. For each point, assert the old asset ID and payload remain unchanged, live-resource count returns to its expected baseline, and no resource is released twice.

Do not simulate allocation failure by exhausting the host machine. Use a deterministic injectable mock loader. The tests should fail at a named step and produce a reproducible state transition. Later you can test a real allocator or device boundary in an isolated integration test.

A strong guarantee can cost extra memory because old and new resources coexist before commit. That is a tradeoff to quantify, not a reason to abandon failure correctness silently. A memory-constrained engine might publish a weaker guarantee with an explicit fallback state.

Transactional replacement of asset A by asset B

The current owner holds A. Construct candidate B independently. If B's validation throws, candidate cleanup releases B and current still owns A. If validation succeeds, swap the two owners. Current now owns B; the candidate temporarily owns A and releases it at scope exit.

At peak, both assets are live. After completion, only the selected current asset remains live. Moving an owner into a second variable transfers its release obligation but does not load a second underlying asset.

Test move assignment into a destination that already owns C: C must be released exactly once before the destination takes the new resource. Self-move must not duplicate ownership or corrupt the wrapper's invariant.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_12_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
void replace_asset(AssetOwner& current, Loader& loader) {
    AssetOwner candidate = loader.load_and_validate();
    current.swap(candidate); // documented nonthrowing commit
}
```

Common failure modes

A moved-from object is not universally empty. Declaring a destructor can affect implicitly generated move operations. `noexcept` is a promise, not a request for the compiler to make code safe. Testing only move construction misses move-assignment leaks.

Practical assignment

Original requirement: Implement a move-only mock resource and transactional asset replacement. Pass: copying is prohibited, moving does not double-release, and injected load failure preserves the old asset.

1. Implement a move-only mock owner or explain a rule-of-zero alternative.
2. Add compile-time tests proving copying is prohibited and move construction is supported.
3. Build a transactional replacement function and inject failures before commit.
4. Measure the maximum simultaneous live-resource count and state the memory tradeoff.

Required cases: copy constructible is false; move leaves exactly one owning handle; move assignment releases prior destination; self-move remains valid; load/validation failure preserves old asset; successful replacement releases old asset once.

Evidence to submit: Ownership transfer trace, compile-time constraints, failure matrix and a statement of basic/strong/no-throw guarantees.

Independent transfer: A reload operation updates both an asset and an ID index. Show why swapping only the asset may not provide the strong guarantee for the whole repository.

Assigned primary reading: R11: class special members, rule of zero and exception safety.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does std::move itself guarantee that an object is moved?
2. What is the strong exception guarantee?
3. Why prefer the rule of zero?
4. Why might transactional replacement need more peak memory than destructive replacement?
5. What happens when an exception escapes a function declared `noexcept`?

12.3 — Interfaces, composition and polymorphism

Learning outcome: Choose abstractions according to substitutability and measured cost.

An interface is a behavioral promise

Two implementations are interchangeable only when they satisfy the same observable contract. A renderer interface might accept a bounded view of visible units and emit a frame without changing simulation state. Whether the backend prints text or records commands is secondary; input validity, ordering and error behavior must match.

Substitutability is stronger than matching method names. A recording renderer that silently sorts units changes observable order if the contract promises input order. A backend that stores a borrowed span after the call changes the lifetime obligation. Specify whether the method consumes the data synchronously or retains owned copies.

Keep interfaces narrow enough to test. A renderer should not need to own the entire game world just to draw a few unit positions. This lesson uses a non-owning renderer reference for one immediate call and a separately owned backend. The ownership arrangement remains visible. [R11: I, C]

Composition and runtime polymorphism solve different problems

Composition assembles behavior from members or collaborators: a simulation uses a renderer and a repository. Inheritance expresses an interface relationship or carefully designed implementation relationship. Prefer a clear has-a relationship over an inheritance hierarchy built merely to share fields.

Runtime polymorphism through a virtual interface supports substituting a backend whose concrete type is selected at runtime. A base intended for deletion through a base pointer needs suitable virtual destruction. An alternative is a protected nonvirtual destructor when deletion through the base is forbidden; the design must make that policy enforceable.

Do not claim virtual calls are always too slow or always free. Their cost depends on dispatch, inlining opportunities, data access and call frequency. A single virtual call submitting a batch differs from one virtual call per component in an inner loop. The next lesson introduces static generic interfaces and data-layout experiments.

A recording implementation makes tests observable

A recording renderer copies each submitted command into an owned vector. Tests can compare the resulting command sequence without opening a window. A console renderer writes the same logical commands to an output stream supplied by its caller. Both are checked against the same expected IDs and positions.

The recording implementation is not a fake that can ignore the contract. It should preserve ordering, reject the same invalid inputs when validation belongs to this boundary and document any intentional difference such as output destination. Tests that pass only because a mock behaves more permissively can hide integration defects.

Inject an output stream rather than writing directly to a global console object. That permits an in-memory stream in a test and clarifies which subsystem owns the output destination. Avoid a global service locator for this small example; explicit parameters make dependencies reviewable.

Separate architecture decisions from fashion

Write a brief decision record: the required variability, candidate designs, ownership model, error behavior and measurements that could overturn the choice. Candidates might include a virtual interface, a template parameter, a tagged variant or a direct concrete class. No single mechanism wins all requirements.

For a classroom backend selected at runtime, a small virtual interface is reasonable. For a fixed inner-loop mathematical kernel, a constrained template may expose optimization opportunities. For a closed set of alternatives, a variant can make exhaustive handling visible. Choose from the constraints rather than using every abstraction feature in one example.

The resource-manager project asks for a written comparison of composition and inheritance. Ground that comparison in your actual code: which state changes together, which dependencies have independent lifetimes, and which tests become simpler when responsibilities are separated?

One simulation, two renderers

Create units in the order [7,2]. The contract says `render` submits commands in that order and does not retain borrowed unit storage. A console backend writes `7:2,3` then `2:4,5`; a recording backend stores two equivalent command values. Modifying the source unit vector after the call cannot change already recorded commands.

Own a backend through `std::unique_ptr<Renderer>`, call through `Renderer&`, and let destruction occur through the base owner. The base's virtual destructor ensures the derived cleanup runs. The simulation itself neither allocates nor deletes the borrowed renderer reference.

A test checks ordering and copied values, not just the number of calls. That is what makes substitutability observable.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_12_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
struct Renderer {
    virtual ~Renderer() = default;
    virtual void draw(std::span<const Unit> units) = 0;
};
void frame(Renderer& renderer, std::span<const Unit> units) {
    renderer.draw(units); // borrowed for this call only
}
```

Common failure modes

An interface without lifetime rules is incomplete. A mock that ignores errors is not equivalent behavior. A base destructor need not always be virtual, but deletion through a base must be deliberately safe or deliberately forbidden.

Practical assignment

Original requirement: Create interchangeable console and recording renderers behind one small interface. Pass: both satisfy the same contract, tests run without display output, and ownership/destructor behavior is explicit.

1. Write a one-page renderer contract before either implementation.
2. Implement console and recording backends with the same unit ordering and synchronous-borrow policy.
3. Test both through the interface with empty and multiple-unit inputs.
4. Write an architecture decision comparing runtime polymorphism with composition and a static alternative.

Required cases: empty input emits no commands; two units preserve input order; recorded commands own copied data; derived cleanup observed through base ownership; simulation does not mutate renderer ownership.

Evidence to submit: Interface contract, two backend implementations, shared contract tests and design decision record.

Independent transfer: A backend queues rendering asynchronously. Amend the contract to transfer owned command data or guarantee a valid lifetime until completion.

Assigned primary reading: R11: Interfaces, Class Hierarchies and composition guidance.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What does substitutability require?
2. Why can a polymorphic base need a virtual destructor?
3. Why not put every responsibility into one engine class?
4. A backend retains a span although the interface promises use only during the call. Is matching output today sufficient?
5. Why might one virtual call per batch have a different cost from one per entity?

12.4 — Templates, concepts and data-oriented choices

Learning outcome: Use generic code without hiding costs or exploding complexity.

Generic code still needs a semantic contract

A function template describes a family of functions. A C++20 concept constrains which type expressions are accepted by an interface and can improve diagnostics. It does not automatically prove every runtime property. A type meeting an ordered concept can still supply problematic values such as NaN in floating-point use.

For a teaching clamp, require an arithmetic domain you can explain, validate that the upper bound is not below the lower bound, and choose a nonfinite-value policy. The included example constrains an integer clamp to integral types and tests invalid bounds explicitly. This avoids silently presenting a generic floating-point contract it has not specified.

A template definition normally must be visible where it is instantiated, which is why many small generic functions live in headers. That can increase compilation work and expose more implementation detail. Explicit instantiation and separate implementation strategies exist, but are beyond this introductory example. [R11: T; P4-01]

Static and dynamic choices have different costs

A template-selected operation can allow specialization and inlining when the compiler sees the concrete types. It can also generate multiple versions, increase binary size and make diagnostics difficult. Runtime dispatch can centralize code and choose behavior at runtime, but may limit optimization at some call sites.

Do not equate the absence of virtual functions with good design. A deeply nested generic engine that takes minutes to compile and is hard to debug has costs outside its frame time. Record runtime, build time, binary size and maintenance constraints when those are relevant to your decision.

The goal is to express a simple contract without hiding its costs. A short constrained function is a better starting point than a universal entity framework. The module project should remain small enough that the learner can explain every ownership path.

Array of structures and structure of arrays

An array of structures stores complete entities next to one another: position, velocity and other fields for entity 0, then entity 1. A structure of arrays groups the same field across entities. If a hot loop reads only x position and x velocity, grouping those fields can reduce unrelated data traffic. If work uses every field of one entity together, the tradeoff can differ.

Both layouts can preserve exactly the same simulation semantics. Keep entity count, arithmetic, update count and output validation equivalent. A benchmark that updates fewer fields in one version is not comparing layout alone. Allocations, initialization and result validation should be placed consistently outside or inside the timed region according to the declared experiment.

The provided small benchmark emits CSV and a checksum. Its output is a local measurement, not a universal ranking. Real hardware adds cache levels, prefetching, vectorization and bandwidth effects studied in later volumes. [R15; R11: Per]

Design a measurement that can be contradicted

Write a hypothesis before timing: for this x-only update over this number of entities, layout A might move less unrelated data than layout B. Record compiler, optimization flags, CPU identity, input count and repetitions. Run multiple samples, alternate order when practical and inspect the distribution rather than reporting a single best number.

Use a monotonic clock for elapsed intervals. Make the result observable through a checksum so the compiler cannot discard an unused calculation. Check the checksum against a known or independently computed result. Do not add `volatile` to every field as a substitute for proper experimental design: that changes the memory-access semantics under study.

A small timing difference near noise is an inconclusive result. That is valid engineering evidence. Follow up with larger representative inputs and profiling, while retaining the correctness tests. Never promote an optimization whose output is wrong simply because its measured time is low.

Clamp contract and layout experiment

For `clamp_int(12,0,10)`, the result is 10. For `clamp_int(-3,0,10)`, it is 0. For invalid bounds (10,0), the function rejects the request rather than returning an arbitrary result. These tests cover the contract independently of the concept's compile-time acceptance.

The layout experiment initializes n entities with x = 0 and vx = 1. Each of r updates adds vx to x. Both implementations must finish with checksum n*r. Start with a small count that can be checked by hand, then scale the same operation. Record times only after the result passes.

A faster result for one x-only workload does not show that the layout is better for collision queries, serialization or every target CPU.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_12_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
template <std::integral T>
T clamp_int(T value, T low, T high) {
    if (high < low)
        throw std::invalid_argument("reversed bounds");
    return value < low ? low : (high < value ? high : value);
}
```

Common failure modes

A concept does not validate runtime bounds. Two layouts doing different work are not a controlled comparison. An optimized-away loop produces a timing, not useful evidence. An inconclusive measurement is better than a fabricated performance rule.

Practical assignment

Original requirement: Write a constrained generic clamp or interpolation function and benchmark two simple entity layouts. Pass: test contracts, record compiler/build mode and explain why the result is workload-specific.

1. Implement a constrained integer clamp and test its accepted types and bound policy.
2. Implement equivalent array-of-structures and structure-of-arrays updates.
3. Collect repeated Release measurements with a verified checksum and saved CSV.
4. Explain the result using only what the workload and measurement support; list one workload that could reverse it.

Required cases: below, within and above bounds; equal lower and upper bounds; reversed bounds rejected; same checksum for both layouts; zero entities handled; multiple sizes and repeated samples recorded.

Evidence to submit: Constrained function, contract tests, benchmark manifest, raw CSV and a bounded interpretation of the measurements.

Independent transfer: The hot loop changes from x-only integration to accessing every field of a single selected entity. Predict why the layout conclusion might change, then design a fair test.

Assigned primary reading: R11: Templates and Generic Programming; R15: memory hierarchy motivation.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What does a concept constrain?
2. Is structure-of-arrays universally superior?
3. Name a cost of templates beyond runtime.
4. Does satisfying a concept prove runtime bounds are valid?
5. Why must both layout variants produce an observable checked result?

Module 13 — Data structures and their costs

Prerequisites: Modules 06 and 11-12.

Project: Implement educational versions of a dynamic array, queue, hash table and heap, each with invariants and tests. Compare them to standard containers without replacing standard code in production merely for practice.

Primary references: R13, R06. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

13.1 — Complexity, arrays and linked structures

Learning outcome: Predict asymptotic work while recognizing locality and constants.

Define the cost model before using Big O

Asymptotic analysis describes how an operation count grows with input size under a chosen model. O(n) is an upper-bound class, not an exact runtime or a claim that all linear algorithms are equally fast. When you can justify both upper and lower growth bounds, Theta notation expresses the tighter classification. Separate worst-case, average-case and amortized claims.

For a contiguous array, indexing is constant-time under the ordinary random-access model, scanning n elements is linear, and inserting at the beginning requires shifting elements. For a linked list, insertion at an already-known position can be constant-time, but finding that position may be linear. Include the lookup cost in the actual workload.

A game scheduler that repeatedly searches a linked list for an insertion point does not get constant-time insertion overall merely because the final pointer rewiring is short. Write the complete operation: locate, validate, allocate, mutate and update any indices. The model should include the dominant work you are trying to compare. [R13: analysis, arrays; R06: sums]

Why geometric growth is amortized constant time

A growable array keeps size <= capacity. When full, it allocates more storage, copies or moves the existing elements and then appends. If capacity increases one slot at a time, appending n elements copies 0+1+…+(n-1) = n(n-1)/2 old elements. That is quadratic total copy work.

If capacity doubles from 1 to 2 to 4 and so on, the copy counts form a geometric series. Before n appends, the total number of relocated old elements is less than 2n, with details depending on initial capacity. Add the n new writes and the total remains linear. Therefore the amortized cost per append is constant under this model, even though a particular growth append costs linear time.

Amortized analysis does not assume random input. It spreads the expensive operations across the sequence. It also does not guarantee a short latency for every frame. A real-time path might reserve a known bound before gameplay or use another allocation strategy to avoid a growth spike.

A deliberately restricted educational buffer

The lab's growable buffer stores integers, not arbitrary C++ objects. It uses an owning array and tracks size, capacity and relocation count. Restricting the element type lets you focus on the size/capacity invariant before learning raw storage, placement construction, partial element construction and allocator traits.

Before expanding capacity, check that the growth calculation and byte count remain within the chosen teaching limit. Allocate the candidate array first, copy existing integers, then commit the new storage. A failed allocation must not lose the original owner. This is the same prepare-then-commit pattern introduced in resource management.

Bounds-checked access rejects an index equal to size, even if that index is below capacity. Allocated storage is not the same as a valid logical element. Test empty access, every growth boundary and value preservation after relocation. Do not advertise this restricted buffer as a production replacement for `std::vector`.

Locality explains why equal asymptotic work can differ

A contiguous scan touches nearby elements. A linked traversal follows pointers whose targets may be scattered. Both scans are O(n), but their constants and interaction with the memory hierarchy can differ substantially. That is a hypothesis to measure on the real workload, not a universal numerical claim.

Collect relocation counts separately from elapsed time. Counts can validate the derived model exactly. Timings also reflect allocation, cache behavior, compiler decisions, clock noise and other activity. A timing curve alone does not prove the asymptotic bound, especially over a narrow range of small inputs.

For the module project, compare your educational structures with standard containers using the same logical operation trace. Keep the custom implementation because it exposes invariants and costs, not because writing it yourself makes it automatically faster or safer. [R15; R13]

Predict eight appends

With exact-fit growth, capacities are 1,2,3,4,5,6,7,8 and relocation count is 0+1+2+3+4+5+6+7 = 28. With doubling, capacities reach 1,2,4,8 and relocation count is 0+1+2+4 = 7. Both store the same eight final integers.

At the ninth append, doubling relocates eight elements in one operation. This is consistent with amortized constant time: an individual append can still be expensive. The exact-fit strategy relocates eight as well, but has already paid much more over the preceding sequence.

Fill a table with size, capacity, relocation count for each append before running. Compare the counters exactly; compare elapsed time only after both versions preserve all values.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_13_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// Cost model for exact-fit growth over n appends:
// moved = 0 + 1 + ... + (n - 1)
//       = n * (n - 1) / 2
// Compute this formula only after checking its numeric range.
// Doubling copies 1 + 2 + 4 + ... < 2*n old elements.
```

Common failure modes

O(1) insertion at a known linked node does not include finding the node. Amortized constant time does not bound worst-case frame latency. Size is the access boundary; spare capacity is not permission to read an element.

Practical assignment

Original requirement: Implement a growable integer buffer for study. Count copied elements under doubling and one-at-a-time growth. Pass: derive the growth behavior and separately measure elapsed time without equating it to the proof.

1. Implement an integer-only buffer with size/capacity invariants and two growth policies.
2. Instrument relocated-element counts and predict the sequence for n = 8 and n = 9.
3. Test value preservation, empty access, exact growth boundaries and a chosen capacity budget.
4. Compare counts analytically, then collect separate Release timings against std::vector.

Required cases: empty buffer has size 0; out-of-range access rejected; n=8 exact-fit moves 28; doubling moves 7; n=9 doubling moves 15; values preserved through growth; capacity limit checked before arithmetic.

Evidence to submit: Growth trace, geometric-series argument, invariant tests and timing results separated from the proof.

Independent transfer: A game must never allocate during a frame. Explain why amortized O(1) is insufficient and design a capacity-planning policy with an explicit failure result.

Assigned primary reading: R13: arrays, cost models and asymptotic analysis; R06: sums.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does O(n) specify an exact runtime?
2. What is the total 1+...+(n-1) proportional to?
3. How does amortized analysis differ from random-input average-case analysis?
4. Can an amortized O(1) append take O(n) time on one call?
5. Why does timing two small input sizes not prove an O(n log n) bound?

13.2 — Stacks, queues and ring buffers

Learning outcome: Match ordering semantics to the intended workflow.

Ordering is a semantic requirement

A stack removes the most recently inserted item first: last in, first out. A queue removes the oldest inserted item first: first in, first out. A priority queue removes according to priority rather than arrival order. Choose from the meaning of the work: undo history, event ingestion and pathfinding frontiers need different rules.

A queue that drops or reorders commands changes simulation behavior. Specify whether capacity is bounded, whether insertion can fail, and what happens when the structure is empty. Do not let overwriting an unread command become an accidental overflow policy.

This lesson's queue is bounded and single-threaded. Push returns false when full; pop returns an optional integer when nonempty. The caller decides whether to retry, reject the command or apply a documented higher-level policy. The data structure does not silently choose gameplay behavior. [R13: elementary structures]

A ring buffer reuses a fixed allocation

Store a fixed-capacity array, a head index naming the next readable slot, and a count of live entries. The next write index is `(head + count) % capacity`. A push writes there and increments count; a pop reads at head, advances head modulo capacity and decrements count.

Count distinguishes empty from full: count = 0 means empty and count = capacity means full. Another valid design reserves one slot so equal read and write indices always mean empty. Do not mix the two conventions. The usable capacity and full test would disagree.

Reject zero capacity or use a representation whose zero-capacity case avoids modulo by zero. The included compile-time-capacity ring requires capacity > 0. Capacity one is a valuable boundary test because every successful operation wraps the index.

State the invariant and trace the transitions

For the count-based design, require 0 <= count <= capacity and head < capacity. Exactly count logical entries are readable in FIFO order beginning at head. Slots outside that logical range do not contain live queue entries, even if they contain old integer bits from earlier operations.

A failed push must leave head, count and existing contents unchanged. A failed pop must not read a slot. A successful pop returns the old head value before updating the indices. These statements are short enough to inspect in every branch of the implementation.

Trace capacity three: push A, B, C; reject D; pop A; push D; pop B, C, D. The physical order in the backing array can be D, B, C while the logical order is B, C, D. Tests must verify logical order, not assume that array index zero is always the front.

Single-thread correctness is not concurrency correctness

A ring buffer with correct sequential transitions can still have data races when a producer and consumer access it concurrently. Atomic indices, ordering of payload publication, ownership of each slot and wraparound reasoning become separate requirements. A random stress test is not a substitute for a memory-order argument.

Do not add a few atomics and label this exercise lock-free. The current module deliberately stays single-threaded. Later operating-system and concurrency lessons will revisit the queue with the required memory-model background.

For this lab, compare thousands of deterministic push/pop operations against `std::deque`. After every operation compare success or absence, logical size and returned values. Include more than three times the capacity in operations, repeated empty/full transitions and capacity one. This gives much stronger evidence than one push followed by one pop.

Capacity-three trace

After push 10,20,30: head = 0 and count = 3. Another push fails with no state change. Pop returns 10; head = 1 and count = 2. Push 40 writes at `(1+2)%3 = 0`; count returns to 3. The backing array is [40,20,30], but successive pops return 20,30,40.

When the last item is popped, count becomes zero. Head may be nonzero; that does not mean the queue is nonempty. Emptiness follows the count convention. Never inspect a stale slot to infer whether an item is live.

Draw each operation as a before/after state, then run the same trace against the supplied reference.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_13_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
bool push(int value) {
    if (count_ == Capacity) return false;
    data_[(head_ + count_) % Capacity] = value;
    ++count_;
    return true;
}
```

Common failure modes

Read/write-index equality is ambiguous unless the convention resolves it. Capacity zero makes a naive modulo invalid. Old bits in an empty slot are not a queued item. Sequential tests do not establish thread safety.

Practical assignment

Original requirement: Implement a bounded queue with explicit full/empty results and wraparound tests. Pass: preserve FIFO order through more than three times its capacity in operations and never read an uninitialized slot.

1. Choose and document the count-based full/empty convention.
2. Implement bounded push/pop without dynamic allocation during operations.
3. Hand-trace wraparound at capacity three and test capacity one.
4. Differentially test a long deterministic operation sequence against std::deque and preserve the seed or trace.

Required cases: pop empty -> absent; push full -> false with unchanged state; capacity-one alternating operations; wraparound returns FIFO order; more than 3*capacity operations; no read from a logically empty slot.

Evidence to submit: State-transition table, invariant checks, failure-policy documentation and differential-test trace.

Independent transfer: An audio queue must never block but may drop old samples. Define a different overflow policy and identify which tests must change; do not silently reuse the command-queue contract.

Assigned primary reading: R13: elementary data structures; R16: later concurrency comparison.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Which item leaves first after pushing A then B onto a stack?
2. Why does a ring buffer need a full/empty convention?
3. Does a correct single-threaded queue imply a thread-safe queue?
4. After wrapping, must the front item be stored at array index zero?
5. Why must a failed push leave existing entries unchanged?

13.3 — Hash tables and collision policies

Learning outcome: Explain expected lookup behavior and its assumptions.

Hashing narrows a search; equality decides identity

A hash function maps a key into a finite hash value. Distinct keys can share the same hash. A dictionary must still compare keys using its equality rule. Equal keys must produce compatible hash values; otherwise lookup may search a different bucket from insertion.

In separate chaining, each bucket contains entries whose hash selects that bucket. Lookup computes the bucket and then checks equality among its entries. In open addressing, entries occupy a shared array and collisions follow a probing rule. Both require precise handling of empty states, existing-key updates and resizing.

For a game asset table, decide whether the same ID replaces a value, rejects a duplicate or creates a versioned record. The included educational table uses update semantics: inserting an existing key changes its value without increasing entry count. This is a contract choice, not an automatic property of hashing. [R13: hashing]

Expected constant time has assumptions

For a chaining table with n entries and m buckets, load factor alpha = n/m. Under an appropriate distribution model, expected chain length is related to alpha, so expected lookup work can remain bounded when load is controlled. A terrible hash function can put every key in one bucket and make lookup linear in n.

Hash computation and key equality have costs too. Hashing a long string is not necessarily constant time in the string length. State which input size the complexity claim concerns. Expected O(1) dictionary access usually abstracts or separately accounts for the key-processing cost.

A resizing policy keeps occupancy within a chosen range, but cannot rescue all adversarial hash distributions. The lab includes a constant-hash test specifically to expose correctness under collisions. It should remain correct even when it is deliberately slow. Do not conflate a performance failure with permission to return the wrong value.

Rehashing changes placement, not identity

When bucket count changes, recompute every entry's bucket under the new count. Copying the old bucket array without redistributing entries breaks lookup. Allocate candidate buckets, reinsert or redistribute the existing pairs, then commit the new bucket collection.

For the restricted integer table, copying into candidate buckets makes failure reasoning manageable. A general-purpose table must additionally handle throwing hashes, throwing key/value operations, iterator invalidation and allocator behavior. State these restrictions rather than claiming industrial robustness from a short classroom implementation.

The table invariant is: entry count equals the total number of entries in all buckets, keys are unique under the equality rule, and every stored key resides in its computed bucket. Verify the invariant after insert, update, erase and resize. A test that only checks size can miss a misplaced entry.

Deletion policies differ by collision strategy

In chaining, erasing one matching entry leaves the others searchable. In open addressing, marking a slot completely empty can prematurely stop a probe that should continue to a colliding key. Tombstones or a carefully justified alternative deletion algorithm preserve search continuity.

Choose chaining for the required reference implementation; explain the open-addressing deletion problem on paper. This keeps the implementation scope bounded while still teaching why collision policy affects correctness. An extension can implement tombstones and compare probe counts at different loads.

For the module comparison, feed your table and `std::unordered_map` the same deterministic operations and compare resulting key/value mappings. Do not require the same iteration order: the contract here concerns mappings, not bucket order or a vendor-specific traversal sequence.

Force three keys into one bucket

Use a hash function that always returns zero. Insert (4,40), (9,90) and (14,140). All occupy the same chain. Query 9 by comparing keys until equality succeeds. Update key 9 to 91; size remains three. Erase key 4; queries for 9 and 14 must still succeed.

After resizing, the constant hash still puts all entries in one bucket. Correctness remains intact while the worst-case search remains linear. This experiment separates collision handling from distribution quality.

For ordinary hashing with 30 entries and 10 buckets, alpha is 3, but that average alone does not tell you the maximum chain length. Inspect the distribution as well.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_13_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
for (auto& entry : buckets_[bucket_for(key)]) {
    if (entry.first == key) {
        entry.second = value; // update, not a second key
        return;
    }
}
// append a new pair only when equality found no match
```

Common failure modes

Matching hashes do not establish equal keys. Rehashing is not a raw resize of bucket storage. Erasing an open-addressed slot as empty can break later lookups. A stable logical result does not require stable hash iteration order. The educational map deliberately disables copy and move: blindly moving its bucket vector while copying its size metadata would break the moved-from invariant. A production value-like map needs a correctly specified and implemented special-member policy.

Practical assignment

Original requirement: Implement chaining or open addressing with collision tests and resizing. Pass: deliberately force many collisions, test missing keys and verify that distinct equal-hash keys remain distinct.

1. Implement a restricted chaining table with insert/update/find/erase and controlled resizing.
2. Check the full invariant after every mutation.
3. Force all keys to collide and test equal-hash distinct keys, updates and deletions.
4. Compare deterministic traces against std::unordered_map and report occupancy/probe observations separately from correctness.

Required cases: missing key -> absent; same key update does not increase size; distinct equal-hash keys preserved; erase chain head/middle/tail; all entries survive resize; negative and zero integer keys handled.

Evidence to submit: Collision-policy specification, resize reasoning, invariant checks and differential mapping tests.

Independent transfer: A mod supplies many deliberately colliding asset names. Explain why correctness tests are insufficient for a production resource-exhaustion defense.

Assigned primary reading: R13: hashing and dictionary data structures.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does a matching hash prove two keys are equal?
2. What is the load factor for 30 entries in 10 buckets?
3. Why is expected O(1) not an unconditional guarantee?
4. Why can clearing an open-addressed slot to empty break lookup?
5. Does load factor 1 guarantee every bucket contains exactly one entry?

13.4 — Trees, heaps and priority queues

Learning outcome: Preserve structural invariants and choose the right query behavior.

Trees answer different questions

A binary search tree orders keys so a search can discard one side at each comparison. Its operation cost depends on height. An unbalanced tree can become a chain after ordered insertion, with linear height and linear search. A balanced search tree controls height to support logarithmic search and updates under its algorithm's assumptions.

A heap solves a different problem: efficiently access an extreme-priority element. A binary min-heap requires each parent to be no greater than its children. It does not order siblings or provide efficient arbitrary-key search. An array satisfying the heap invariant is not a fully sorted array.

Choose from the query contract. An ordered asset index needing range queries suggests an ordered search structure. A pathfinder repeatedly extracting the smallest tentative cost suggests a priority queue. A structure's name is not evidence that it supports every kind of ordering efficiently. [R13: trees, heaps]

An array represents a complete binary heap

In a zero-based array, the parent of a nonroot node i is `(i-1)/2`; children are `2*i+1` and `2*i+2` when those indices exist. Check existence before access and avoid overflow in index arithmetic for unrestricted sizes. The teaching heap has a bounded input scope and uses loop conditions that avoid forming invalid children.

To push, append at the end and repeatedly exchange with the parent while the new item is smaller. To pop, remove the root, move the last element into its place, shrink the array and sift down through the smaller child. Each path has logarithmic height in a complete binary tree.

Popping a one-element heap needs special handling: after removing that element, there is no root to repair. Popping an empty heap returns absence under our contract. Duplicates are allowed and do not violate parent <= child ordering.

Tie-breaking is observable behavior

Two events can have equal priority. A queue may return them in an unspecified relative order unless the comparator defines a secondary key. For reproducible processing, use a stable sequence number or another explicit tie-breaker. The complete comparison must satisfy a strict weak ordering for standard ordering algorithms.

Do not use <= as a less-than comparator: an item would compare less than itself. For a tuple (priority, sequence), ordinary lexicographic less-than gives a clear total ordering when both components have ordinary integer order. If sequence numbers can wrap, define a bounded lifetime or a checked exhaustion policy.

Floating-point NaN complicates comparisons. A queue of finite validated costs can avoid that domain problem; otherwise specify a total-order policy rather than assuming ordinary < handles every value coherently. The pathfinding reference uses bounded nonnegative integer costs.

Test the invariant, not only the final answer

After every push and pop, scan the array and verify each child's priority is not less than its parent's. This O(n) test instrumentation is intentionally outside a claim about production push/pop complexity. It provides detailed failure localization during learning.

Use a sorted vector or multiset as a small independent oracle. Generate a deterministic sequence with duplicate and negative integer priorities, compare each extracted value and then check the heap invariant. A test that inserts random values and only checks final sorted output can reveal a defect, but checking every mutation tells you where it started.

Also explain why a min-heap cannot answer a general sorted range query as cheaply as a balanced ordered tree. The module project should document the data structure you did not choose and the requirement that would justify it.

Push 7, 2, 5, 1

After 7, the heap is [7]. Push 2 and exchange with 7: [2,7]. Push 5: [2,7,5]. Push 1 at index 3, exchange with 7 and then 2: [1,2,5,7]. This happens to look sorted, but that is not required.

Pop 1, replace it with 7, then choose the smaller child 2 and exchange: [2,7,5]. This is a valid heap and not a sorted array. The next pop is still 2. Continue and expect 5 then 7.

For equal priorities, attach insertion sequence numbers and predict their exact extraction order. Without such a rule, do not write a test that assumes stable ordering the container never promised.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_13_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
for (std::size_t child = 1; child < heap.size(); ++child) {
    const std::size_t parent = (child - 1) / 2;
    CHECK(!(heap[child] < heap[parent]));
}
```

Common failure modes

A heap is not globally sorted. A BST is not automatically balanced. A comparator using <= violates irreflexivity. An invariant scan adds testing cost; do not accidentally include it in a benchmark claiming logarithmic operation timing.

Practical assignment

Original requirement: Implement a binary min-heap with push/pop and randomized comparison to a sorted reference. Pass: validate the heap invariant after every operation and test duplicate priorities.

1. Implement a binary min-heap with explicit empty behavior and a size budget.
2. Validate parent/child order after every mutation.
3. Compare push/pop traces with a sorted independent oracle, including duplicates.

4. Add an event-priority tie-breaker and compare heap requirements with balanced-tree requirements.

Required cases: empty pop -> absent; one-element pop leaves empty; ascending and descending insertions; duplicate priorities retained; negative priorities ordered correctly; invariant after every randomized operation.

Evidence to submit: Index diagram, sift traces, invariant checker, differential tests and a data-structure decision table.

Independent transfer: A scheduler now needs all events whose times lie in an interval, not only the next event. Reconsider the structure and explain the changed query cost.

Assigned primary reading: R13: binary search trees, balanced trees and heaps.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does a heap provide a fully sorted array?
2. What can the height of an unbalanced n-node search tree become?
3. Why define a tie-break rule in a priority queue?
4. Is [2,7,5] a valid min-heap?
5. Why is `a <= b` not a suitable less-than comparator?

Module 14 — Algorithms for maps, simulation and tools

Prerequisites: Modules 06, 09 and 13.

Project: Build a pathfinding visualizer and asset-dependency planner. Provide correctness arguments, unreachable cases, complexity analysis and measured comparisons on multiple map families.

Primary references: R13, R06. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

14.1 — Searching, sorting and algorithm selection

Learning outcome: Choose an algorithm from its preconditions and the data distribution.

Choose from preconditions, not algorithm prestige

Binary search is useful when the range is ordered under the same comparison used by the search. Repeatedly halving an unsorted range has no correctness basis. Sorting first has a cost, so the best choice depends on whether there is one query or many, how often data changes and whether the sorted representation is useful elsewhere.

Linear search can be entirely appropriate for a short or frequently changing list. A game frame's actual workload may be dominated by memory access or maintaining an index rather than the comparison count. Start with a correct simple baseline and identify the measured bottleneck.

For sorting, compare not only asymptotic behavior but stability, auxiliary storage, input distribution and the cost of comparisons or moves. An educational insertion sort makes shifts visible and can exploit nearly sorted input; merge sort provides a predictable divide-and-combine structure. Standard algorithms remain the production baseline. [R13: searching, sorting]

Half-open intervals prevent many off-by-one errors

The reference lower-bound search maintains a candidate interval [lo,hi), with 0 <= lo <= hi <= n. The answer is the first position whose element is not less than the target, or n if all elements are less. Choose `mid = lo + (hi-lo)/2` to avoid adding two potentially large indices.

If `a[mid] < target`, the answer must lie after mid, so set lo = mid+1. Otherwise the answer lies at mid or earlier, so set hi = mid. Each iteration shrinks the interval. When lo == hi, that position is the lower bound.

This lower-bound result is an insertion position, not automatically a successful membership result. Membership additionally requires position < n and equality under the chosen key comparison. Duplicates are not an error: lower bound returns the first equivalent position.

Correct sorting means order and preservation

A sorted output can still be wrong if it drops an element or duplicates one. Verify that no later element compares less than its predecessor and that the output multiset equals the input multiset. A stable sort additionally preserves relative input order among equivalent keys.

Test stability using records with both a sort key and an original sequence ID. Sorting only raw integers cannot reveal whether equal-key records exchanged order. If a deterministic tie-breaker is part of the comparator, the elements may no longer be equivalent under that comparator, which changes what stability is testing.

For insertion sort, shift larger prior elements and place the held item into the gap. Using a strict comparison when deciding to shift can preserve equal-key order. For merge sort, choosing the left element on equal keys preserves stability across the merge when the recursive halves are stable.

Compare distributions with a reproducible experiment

Use the same inputs for both candidate sorts: empty, one element, duplicates, random values, already sorted, reverse sorted and nearly sorted. Save the random seed or actual input. Validate every output before accepting a timing.

Separate copying input, allocating scratch memory and sorting according to the declared experiment. A fair comparison can include all three as an end-to-end operation, or isolate sorting with prepared storage, but cannot charge setup to only one variant without saying so.

Report the input sizes and distribution, not just an algorithm name and one number. A sort that performs well on nearly sorted data may behave differently on reverse input. The module project asks for multiple map and data families precisely to prevent a favorable single case from becoming an unsupported general claim.

Lower bound of 4 in [1,4,4,9]

Begin lo = 0, hi = 4. Mid = 2, value 4 is not less than 4, so hi = 2. Mid = 1, value 4 is not less, so hi = 1. Mid = 0, value 1 is less, so lo = 1. Now lo = hi = 1, the first position of an equivalent value.

For target 10, the final position is 4, which is a valid insertion position but not an element index. For an empty vector, lo and hi start equal and the loop never reads an element.

For a sorting preservation test, input [2,1,2] must produce [1,2,2], not merely any nondecreasing vector. The output [1,2] is ordered but loses multiplicity.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_14_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
std::size_t lower_bound_index(std::span<const int> a, int x) {
    std::size_t lo = 0, hi = a.size();
    while (lo < hi) {
        const auto mid = lo + (hi - lo) / 2;
        if (a[mid] < x) lo = mid + 1;
        else hi = mid;
    }
```

```
    return lo;
}
```

Common failure modes

An insertion position equal to size is not safe to dereference. Sortedness does not prove element preservation. A stable-sort test needs distinguishable equal-key records. A benchmark without a recorded input distribution is difficult to interpret.

Practical assignment

Original requirement: Implement binary search with a documented interval convention and test empty, one-element, duplicate and missing cases. Compare two sorts on random and nearly sorted inputs. Pass: verify output order and element preservation.

1. Implement lower bound with a written half-open invariant and a separate membership wrapper.
2. Test empty, singleton, duplicate and missing targets against the standard lower-bound algorithm.
3. Implement or inspect insertion and merge sorts, checking order, multiset preservation and stability as applicable.
4. Compare random and nearly sorted inputs with saved data and repeatable Release measurements.

Required cases: empty -> insertion position 0; target below all -> 0; duplicate target -> first equivalent position; target above all -> size; sorting preserves duplicate counts; equal-key records preserve order for stable implementation.

Evidence to submit: Binary-search proof sketch, boundary tests, sorting property tests and distribution-specific comparison.

Independent transfer: The comparator sorts asset names case-insensitively, but search uses case-sensitive comparison. Explain why the search precondition is broken.

Assigned primary reading: R13: binary search, sorting and comparison models.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What precondition does ordinary binary search require?
2. What does stable sorting preserve?
3. Why test element preservation as well as sortedness?
4. What extra check turns a lower-bound position into a membership result?
5. Why test a sort on records containing original sequence IDs?

14.2 — Graph traversal and dependency ordering

Learning outcome: Use graph algorithms with explicit edge semantics.

An edge must have a declared meaning

A graph contains vertices and edges, but the engine supplies their semantics. In a navigation graph, an edge can mean a legal move. In an asset graph, edge A -> B can mean A must be built before B, or it can mean A depends on B. Both conventions are possible and produce opposite scheduling interpretations.

This bundle defines dependency edge prerequisite -> dependent. The topological order must place the edge's source before its destination. Write this convention on the diagram and in the test fixture. A correct graph algorithm applied to reversed semantics can produce a perfectly valid but operationally wrong build order.

Validate vertex IDs at the API boundary. An edge that names an out-of-range vertex is invalid input, not a reason to index outside an adjacency array. Decide how duplicate edges and isolated vertices are represented and ensure indegree counting follows the same representation. [R13: graph representations, BFS, DFS]

Breadth-first search builds distance layers

For an unweighted graph, BFS starts from the source at distance zero, marks it discovered and processes a FIFO queue. Each undiscovered neighbor is marked when enqueued, assigned distance one greater and given a predecessor. Marking at enqueue time avoids repeatedly enqueuing the same vertex through several incoming edges.

The queue processes vertices in nondecreasing distance layers. When a vertex is first discovered, the path uses the fewest edges under the unweighted model. Recover a path by following predecessors from the goal back to the source and reversing the result.

A missing path is a normal result. Source equal to goal yields the one-vertex path if the source is valid. Do not fabricate a predecessor for an unreachable vertex or loop forever while reconstructing it. For weighted costs, fewest edges need not mean least total cost; that is the next lesson.

Topological sorting detects dependency cycles

Kahn's algorithm computes indegrees, starts with all zero-indegree vertices and repeatedly removes one, appending it to the order. Removing a vertex conceptually removes its outgoing edges and reduces each dependent's indegree. Newly zero-indegree vertices become available.

If the output contains fewer than V vertices, some vertices remain in a directed cycle or blocked by a cycle, so no complete topological order exists. Report a cycle/dependency failure instead of silently returning a partial schedule as success. A DFS-based algorithm can additionally construct a cycle witness; that is a useful extension.

Several valid topological orders may exist. To make the teaching output deterministic, choose the smallest available vertex first, using a suitable ordered frontier. That tie-breaking can add a logarithmic factor compared with a plain queue. State whether your complexity claim includes that choice.

Prove the bound for the actual representation

With adjacency lists and a simple queue, a full BFS visits each vertex at most once and scans each outgoing edge at most once, yielding O(V+E) work under constant-time queue operations. An adjacency matrix may require scanning V possible neighbors per vertex and therefore O(V^2) work.

Likewise, basic Kahn sorting with a queue can be O(V+E). A min-heap frontier provides a lexicographically controlled choice but changes the bound to include heap operations. Determinism does not require a heap if insertion and adjacency order are already fully specified; choose deliberately.

The tests validate a topological order by building a position array and checking position[u] < position[v] for every edge u -> v. Comparing only against one expected order can reject a different valid order unless that exact tie-breaker is part of the contract.

Navigation and dependencies use the same structure differently

For edges 0 -> 1, 0 -> 2, 1 -> 3 and 2 -> 3, BFS from 0 reaches 3 in two edges. With adjacency order [1,2], the predecessor of 3 may be 1. Both 0,1,3 and 0,2,3 are shortest paths, but a deterministic rule chooses one.

For the same graph interpreted as prerequisites, a topological order can be 0,1,2,3 or 0,2,1,3. Add edge 3 -> 0 and no topological order exists. BFS still terminates because discovery marks prevent repeated expansion.

Keep the two correctness claims distinct: shortest unweighted path is not the same property as dependency order.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_14_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
queue.push(source);
seen[source] = true;
while (!queue.empty()) {
    const auto u = queue.front(); queue.pop();
    for (const auto v : adjacency[u]) {
        if (seen[v]) continue;
        seen[v] = true; parent[v] = u;
        queue.push(v);
    }
}
```

Common failure modes

Reversed dependency semantics can invalidate a build plan without violating the graph algorithm. Marking only when popped can create repeated queue entries. A partial topological order is not success. Exact tie-breaking can change the complexity bound.

Practical assignment

Original requirement: Implement BFS and topological sorting. Pass: recover a shortest unweighted path, report unreachable goals, reject a cyclic dependency graph and justify the O(V+E) traversal bound for adjacency lists.

1. Specify graph direction, vertex domain and duplicate-edge policy.
2. Implement BFS with predecessor reconstruction and explicit unreachable results.
3. Implement Kahn topological sorting and reject cyclic inputs.
4. Validate path edges and every dependency relation, then justify the bound for your chosen frontier and representation.

Required cases: source equals goal; isolated unreachable goal; diamond graph with two shortest paths; directed cycle; self-loop cycle; disconnected DAG includes all vertices; invalid vertex rejected.

Evidence to submit: Annotated graphs, predecessor traces, order validator, cycle tests and representation-specific complexity argument.

Independent transfer: A planner uses a priority queue to choose the alphabetically smallest ready asset. Update its complexity analysis instead of retaining an unqualified O(V+E) claim.

Assigned primary reading: R13: graph representations, BFS, DFS and topological sorting.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. When does BFS find a shortest path by edge count?
2. Can a directed graph with a cycle have a topological order?
3. Why mark visited nodes?
4. Why mark a BFS vertex discovered when enqueuing rather than repeatedly when dequeuing?
5. How can you test a topological order without demanding one particular valid sequence?

14.3 — Dijkstra, A* and admissible heuristics

Learning outcome: Build a correct pathfinder before optimizing its queue.

Shortest path requires a precise cost model

Dijkstra's algorithm applies to nonnegative edge costs. Maintain the best known distance g[v], initialize the source to zero and use a min-priority queue. When an edge u -> v with cost w offers a smaller g[u]+w, update g[v] and its predecessor and push a corresponding queue entry.

The reference uses a lazy queue: a vertex can have several entries, and a popped entry is skipped when its stored cost no longer equals the best known cost. This avoids needing a decrease-key operation in the introductory implementation. It also means queue size and operation counts differ from a one-entry-per-vertex design.

Validate nonnegative costs before searching. Check distance addition against the chosen numeric budget before computing it. An unreachable goal is not represented by adding to an integer maximum as though it were an ordinary distance. Use an explicit absence or a carefully controlled infinity sentinel. [R13: shortest paths]

A* adds a lower-bound estimate

A* prioritizes f(v) = g(v) + h(v), where h estimates remaining cost. An admissible heuristic never exceeds the true remaining optimal cost. A consistent heuristic satisfies h(u) <= w(u,v) + h(v) on each edge and normally has h(goal) = 0. Consistency supports a simple no-reopening closed-set policy; admissibility alone may require reopening improved nodes.

For a four-neighbor grid with minimum legal step cost c_min >= 0, Manhattan distance times c_min is a lower bound. With inexpensive diagonal moves, ordinary Manhattan distance can overestimate and must be reconsidered. With teleports or special movement rules, geometry alone may not bound route cost.

The teaching search permits improved nodes to be processed again and skips stale entries. With an admissible heuristic and the stated nonnegative-cost/goal conditions, stop when a valid goal entry is selected. For an untrusted arbitrary heuristic, returning the first goal is not a guarantee of optimality.

Construct a counterexample instead of merely warning

Consider S -> G of cost 10, S -> A of cost 1 and A -> G of cost 1. The optimal route costs 2 through A. Set h(A) = 100 and h(G) = 0. After expanding S, A has f = 101 while G has f = 10, so first-goal A* returns the cost-10 route. The heuristic deliberately overestimates and sacrifices optimality.

This is an experimentally checkable failure, not a vague claim that bad heuristics are risky. The lab includes it and compares with Dijkstra's cost 2. Then replace the heuristic with zero: A* becomes the same priority rule as Dijkstra and recovers the optimal result.

A pathfinder must also return a valid route: consecutive cells must be legal neighbors, no blocked cell may appear and the sum of actual edge costs must match the reported cost. Comparing only scalar costs can miss a corrupted predecessor chain.

Measure searches across map families

Test empty/open areas, corridors, obstacles creating detours, disconnected regions and weighted terrain. For small bounded graphs, an independent all-pairs or exhaustive reference can verify distances. Comparing A* only to Dijkstra is useful but can miss a defect shared by their common graph adapter.

Record nodes expanded, edges relaxed, queue pushes, peak queue size and elapsed time. A heuristic can reduce expansions but add per-node calculation cost. A map with one favorable corridor is not representative of all game navigation.

Tie-breaking affects which equal-cost path is returned and can affect expansion count without changing optimal cost. Define a deterministic order for the teaching fixture, then compare costs when several paths are valid. The capstone's correctness requirement is a valid minimum-cost path under the declared model, not a particular visual shape unless explicitly specified.

A weighted-grid baseline and an unsafe heuristic

On a four-neighbor unit-cost grid, from (0,0) to (3,4), Manhattan h = 7. Obstacles can only force an equal or longer route when they remove moves without adding cheaper shortcuts. On a weighted grid with minimum step cost 2, the corresponding lower bound is 14.

Now use the three-node graph S,A,G with costs 10,1,1 described above. Trace the frontier after S: (G,g=10,f=10), (A,g=1,f=101). First-goal A* with the bad heuristic returns 10; Dijkstra returns 2. Preserve this as a regression test showing the limitation of an assumption, not a failure to repair by hiding the test.

For every returned path, recompute its cost from the graph instead of trusting the stored scalar.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_14_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// Lazy-priority-queue relaxation, after checking cost bounds:
if (candidate < distance[v]) {
    distance[v] = candidate;
    parent[v] = u;
    frontier.push({candidate + heuristic(v), candidate, v});
}
// On pop: ignore entries whose stored g != distance[node].
```

Common failure modes

Negative edges violate ordinary Dijkstra assumptions. An overestimating heuristic can return a suboptimal first goal. A permanent closed set can be wrong for inconsistent heuristics without reopening. A correct cost alone does not prove the reconstructed route is valid.

Practical assignment

Original requirement: Implement Dijkstra and A* against the same grid model. Pass: compare path costs on many small maps, include no-path cases and demonstrate a deliberately overestimating heuristic that can sacrifice optimality.

1. Define neighbor rules, blocked cells, edge costs, distance bounds and tie-breaking.
2. Implement Dijkstra and A* with stale-entry handling and an explicit reopening policy.
3. Compare costs and independently validate routes over multiple small map families.
4. Demonstrate the overestimating-heuristic counterexample and record expansion/queue metrics without claiming universal speedup.

Required cases: start equals goal; unreachable region; weighted detour cheaper than direct route; zero-cost edge allowed; negative edge rejected; overestimating heuristic gives 10 versus optimal 2; A* with h=0 agrees with Dijkstra.

Evidence to submit: Cost-model specification, path validator, small-map oracle, counterexample trace and comparative metrics.

Independent transfer: Add diagonal movement with cost 1 while orthogonal movement also costs 1. Replace Manhattan with a valid lower bound and justify it.

Assigned primary reading: R13: shortest paths; R20: game navigation context.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What edge-cost condition does ordinary Dijkstra require?
2. What is Manhattan distance from (0,0) to (3,4)?
3. Why can inconsistent heuristics require reopening nodes?
4. For unit-cost diagonal and orthogonal moves on an unobstructed grid, why can Manhattan overestimate?
5. Why skip a priority-queue entry whose stored g differs from the current best distance?

14.4 — Dynamic programming, complexity limits and approximation

Learning outcome: Recognize repeated subproblems and justify a practical solution.

A state must contain enough history

Dynamic programming reuses solutions to overlapping subproblems. The state must retain everything needed for future decisions without carrying irrelevant history. For 0/1 knapsack, state (i,b) means the best value obtainable from the first i items with capacity b. Remembering only the remaining capacity while recursively choosing arbitrary items can accidentally allow reusing an item.

For item i with weight w_i and value v_i, compare excluding it with including it when it fits. The recurrence is best(i,b) = max(best(i-1,b), v_i + best(i-1,b-w_i)). The base row for no items has value zero. This recurrence explicitly permits each item at most once.

The proof is a case split on an optimal solution: it either excludes the last item or includes it, leaving a valid subproblem over earlier items. The recurrence is not justified merely because the resulting table looks plausible on one input. [R13: dynamic programming]

Evaluation order changes the problem if chosen wrongly

A full two-dimensional table stores every prefix/capacity state. A one-dimensional optimization can reuse the previous row, but for 0/1 knapsack it must update capacities in descending order. Ascending order can read a value updated using the current item and then use the same item again, solving a different unbounded-choice problem.

Use positive integer weights and nonnegative bounded integer values for the teaching implementation. Define a capacity budget before allocating a table and check value additions. Zero-weight and negative-value generalizations require an explicit contract rather than accidental behavior.

The included reference keeps the recurrence readable and compares with exhaustive search for small n. Reconstruction of chosen items is an extension that must verify each item appears at most once and that total weight and value match the reported optimum.

Pseudopolynomial is not polynomial in input bit length

The ordinary integer-capacity knapsack dynamic program uses O(nB) states or transitions up to constant factors, where B is the numeric capacity. If B is encoded in binary, its input length is proportional to log B. Therefore O(nB) is pseudopolynomial, not a general polynomial-time result in the bit length of the instance.

NP-completeness refers to a decision-problem family under formal reductions; it does not mean every individual instance is intractable. An optimization problem is usually described as NP-hard rather than casually treating its output as a yes/no language. Likewise, polynomial complexity does not guarantee that an algorithm fits a 16-millisecond frame budget.

Choose a bounded exact solver for small instances, a precomputed result when inputs change rarely, or an approximation/heuristic with a clear quality evaluation. Do not claim an approximation guarantee from a handful of good outputs. A formal guarantee needs a proof under stated assumptions. [R06; R13]

A simple oracle is a long-term engineering asset

Exhaustive search enumerates all subsets for tiny item counts and computes the best feasible value. Its exponential cost is acceptable when n is intentionally capped in tests. Because its structure differs from the dynamic program, it can reveal recurrence, loop-order and boundary defects.

Generate small deterministic cases, compare both values and validate reconstructed selections when present. Include an item too heavy to fit, a capacity of zero, equal-valued alternatives and a case where a greedy value-to-weight choice loses.

The broader lesson applies to graphics, simulation and tooling: preserve a slow clear baseline for small inputs. Optimize the production path while continuing to compare it against the baseline. Correctness oracles are not obsolete just because they are too slow for the largest runtime workload.

Loadout with capacity five

Items have (weight,value) pairs (2,3), (3,4) and (4,5). Exhaustive feasible choices include the empty set with value 0, each single item with values 3,4,5, and the first two together with weight 5 and value 7. The optimal value is 7.

In the dynamic program, after the first two items, state capacity five compares excluding item two (value 3) with including it plus the best capacity-two solution (4+3=7). Item three cannot improve that result: its value 5 plus the best remaining capacity one gives 5.

For one item of weight 2 and value 3 with capacity 4, descending one-dimensional updates produce 3. Ascending updates can incorrectly produce 6 by reusing the item. Keep this as a targeted regression case.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_14_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// For each positive-weight item in 0/1 knapsack:
for (std::size_t b = capacity; b >= weight; --b) {
    best[b] = std::max(best[b], value + best[b - weight]);
    if (b == weight) break; // avoid unsigned underflow
}
```

Common failure modes

Insufficient state changes the problem. Ascending capacity updates can turn 0/1 knapsack into repeated-item selection. O(nB) is pseudopolynomial in binary-encoded capacity. A heuristic's good example is not an approximation theorem.

Practical assignment

Original requirement: Solve a small 0/1 knapsack instance by exhaustive search and dynamic programming. Pass: compare answers, derive state count, and describe how a heuristic would be evaluated without claiming a guarantee it lacks.

1. Define item and capacity domains and derive the include/exclude recurrence.
2. Implement exhaustive search with a small explicit item-count cap.
3. Implement dynamic programming, checking capacity allocation and value arithmetic.
4. Differentially test small instances and compare one heuristic without overstating its guarantee.

Required cases: no items -> 0; capacity zero -> 0; all items too heavy -> 0; (2,3),(3,4),(4,5), B=5 -> 7; single (2,3), B=4 -> 3, not 6; many deterministic small cases agree with exhaustive solver.

Evidence to submit: State definition, recurrence proof, loop-order explanation, oracle comparisons and quality evaluation of a heuristic.

Independent transfer: The loadout now limits both mass and electrical power. Define the new sufficient state and explain the growth in table size before implementing it.

Assigned primary reading: R13: dynamic programming; R06: reductions and asymptotic reasoning.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What two features motivate dynamic programming?
2. Does NP-complete mean no instance can be solved quickly?
3. Why keep a brute-force solver for tiny inputs?
4. Why must one-dimensional 0/1 knapsack update capacity downward?
5. Why is O(nB) called pseudopolynomial when B is binary encoded?

Module 15 — Programming languages, compilers and computability

Prerequisites: Modules 11-14 and discrete proofs.

Project: Complete Gate C: a tested C++ simulation plus a tiny expression language with lexer, parser, evaluator, bytecode execution and resource limits.

Primary references: R14, R06, R11. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

15.1 — Language paradigms and type systems

Learning outcome: Separate language semantics from syntax and implementation habits.

Paradigms describe how a program is organized

Imperative code updates state through a sequence of operations. Functional style emphasizes computing results from explicit inputs, often using immutable values. Object-oriented design groups behavior behind interfaces and ownership boundaries. These are compatible design tools, not mutually exclusive team identities.

A combat loop can imperatively advance time and emit effects while calling a pure damage function. The pure function can be tested with a simple input/output table because it has no hidden dependency on a global clock, random generator or mutable enemy registry. The surrounding loop still needs tests for order and effects.

Purity does not mean every implementation is allocation-free or fast. It means the chosen function's observable result follows its explicit inputs without the forbidden hidden effects in its contract. A game can use a functional core and an imperative shell while preserving native resource ownership.

Type rules reject some mistakes, not all mistakes

A static type checker applies rules before execution; a dynamic check applies at runtime. The distinction is not simply compiled versus interpreted: an interpreter can perform static checks before evaluating, and a compiled program can contain dynamic type checks.

The toy checker has two types, Number and Boolean. Addition requires two Numbers and returns Number. Logical AND requires two Booleans and returns Boolean. Reject Number + Boolean even if a host language would permit some implicit conversion. These are the toy language's rules, not an obligation to copy every C++ conversion.

A well-typed expression can still divide by zero, exceed an application limit or calculate the wrong game balance. A type system proves only what its formal rules establish. Keep type errors separate from syntax errors, runtime domain failures and gameplay requirement failures. [R14: frontend concepts; R11: I]

An ABI boundary needs a smaller vocabulary

A C-compatible interface should use types and declarations understood by a C compiler, with explicit sizes or opaque handles where appropriate. Do not expose `std::string`, references, templates or C++ exceptions across a C header. `extern "C"` controls language linkage when the header is compiled as C++; it does not turn arbitrary C++ data structures into a portable binary protocol.

Specify which side allocates, which side frees, how long borrowed buffers remain valid, whether null is permitted and how errors are returned. A status code plus an output parameter is one possible C design. On failure, leave the output unchanged or set it to a specified safe value; choose one and test it.

Binary compatibility also depends on compiler ABI, platform, calling convention, alignment and build configuration. A shared C header reduces friction but is not a promise that any binary built by any compiler can link with any other. The mandatory C bridge compiles the implementation as C17 and exercises it from C++20 on the same toolchain. [P4-05]

Keep boundary failures inside the host contract

A C++ wrapper can turn a C handle into a unique owner with a custom deleter. The deleter must call the C library's matching destroy function, not a guessed `delete` or unrelated allocator. A failed creation returns the documented status and no owned handle.

If C code calls back into C++, the callback must prevent exceptions from escaping through a boundary whose contract does not support them. Convert failures into the agreed status channel. This lesson's required interface has no callbacks, keeping that risk out of the basic exercise while making the principle explicit.

The hardware path will later add fixed register layouts, interrupts, DMA and platform-specific calling conventions. This mock interface does not access real hardware. It teaches the allocation, representation and error obligations that must be understood before device integration.

One rule, two styles and a type boundary

Rule: subtract armor from incoming damage, but never apply negative damage. With incoming 12 and armor 5, effective damage is 7. An imperative version assigns a temporary, clamps it and subtracts it from hit points. A pure version returns the new hit-point value from explicit inputs. For hit points 20, both produce 13.

The toy checker accepts Number + Number and rejects Number + Boolean. A division expression can still be type-correct with a zero denominator, so runtime domain checks remain necessary.

At the C boundary, a function `ge_apply_damage(int hp, int damage, int* out_hp)` returns a status, rejects negative domain inputs and a null output pointer, and writes only on success. Its C++ wrapper can return an optional value or throw at a C++ boundary after interpreting the status.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_15_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
#ifdef __cplusplus
extern "C" {
#endif
int ge_apply_damage(int hp, int damage, int *out_hp);
#ifdef __cplusplus
}
#endif
```

Common failure modes

Static typing does not prove all gameplay requirements. Interpreted does not imply dynamically typed. `extern "C"` does not make C++ containers C-compatible. Allocation and release must use the same API family.

Practical assignment

Original requirement: Write one rule in imperative and pure-functional styles, then define a small typed C-compatible interface for a mock subsystem. Pass: state allocation ownership and reject type-invalid expressions in a toy checker.

1. Implement the same bounded damage rule in imperative and pure-functional styles and compare a full input table.
2. Implement Number/Boolean type rules for arithmetic and logical operators.
3. Write a C-compatible header with explicit ownership, null and error policies.
4. Use the later C bridge to compile an actual C implementation and call it through a C++ wrapper.

Required cases: damage below armor -> zero effective damage; damage exceeds remaining HP -> clamp at zero; negative domain input rejected; Number+Number accepted; Number+Boolean rejected; null output rejected without dereference.

Evidence to submit: Two equivalent rule implementations, type-rule table, invalid-expression tests and a documented C boundary.

Independent transfer: A damage function reads a global random generator. Redesign its interface so a test can reproduce the same result without relying on global state.

Assigned primary reading: R14: language frontend concepts; R11: interfaces and C-style boundary guidance.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does static typing prove all logical requirements?
2. What advantage does a pure calculation offer tests?
3. What must a C-compatible ownership boundary specify?
4. Can an interpreter perform static type checking before execution?

5. Why must a C++ wrapper call the C API's destroy function rather than `delete`?

15.2 — Lexing, parsing and syntax trees

Learning outcome: Convert source text into a structured representation with diagnostics.

Write the language specification first

The bundle's tiny expression language uses bounded signed integer values from -1,000,000,000 through +1,000,000,000. Source integer literals are unsigned decimal digits; unary minus introduces negative values. It supports parentheses and +, -, *, /. Division truncates toward zero. Every intermediate result must remain in range, and division by zero is an error.

Only ASCII whitespace and the declared token characters are accepted. Names are not part of the executable baseline: alphabetic input receives an unknown-name/invalid-token diagnostic. The Number/Boolean checker from Lesson 15.1 is a separate type-system exercise, not a claim that booleans are executable in this arithmetic grammar.

Resource limits are part of the language service contract: at most 16,384 source bytes, 4,096 tokens, 4,096 AST nodes and structural/nesting depth 64 in the reference. These are teaching limits chosen to keep failures manageable, not validated production security limits. Do not use host-language `eval` to execute the input. [R14: lexer and parser; P4-06]

Lexing groups characters; parsing gives structure

The lexer scans left to right and produces Number, Plus, Minus, Star, Slash, LeftParen, RightParen and End tokens. Each token retains a source offset. A sequence of digits forms one number; check the literal limit while accumulating or use a checked conversion. Whitespace is skipped without losing the offsets in the original source.

The parser consumes tokens according to precedence. `expression` handles addition and subtraction, `term` handles multiplication and division, and `primary` handles numbers, parentheses and unary minus. A loop for repeated addition/subtraction makes binary subtraction left-associative. A recursive call for unary minus allows a bounded chain such as `--3`.

Require End after parsing the expression. Accepting a valid prefix and ignoring trailing tokens would let `2 3` appear successful. Report the unexpected token's offset and a useful category. A syntax error is different from a runtime error such as division by zero in a syntactically valid expression.

An arena AST makes ownership visible

An abstract syntax tree stores semantic structure rather than every punctuation mark. A binary node contains an operator, child indices and the operator's source location. A literal node contains its value and location. The reference stores nodes in an owning vector and uses indices, avoiding pointers that would dangle when the vector grows.

Child indices must refer to valid earlier nodes in this constructed tree. Structural depth is computed when a node is created and checked against the limit. A long left-associative chain can have shallow parenthesis nesting but a deep AST, so checking parentheses alone is not enough. The arena's destruction is nonrecursive because its nodes own indices, not nested child owners.

Do not keep a reference to a vector element while appending another AST node. Retrieve by index after mutation. This is a direct application of the native lifetime lessons, now inside a compiler frontend.

Evaluation follows the tree and the specified arithmetic

Evaluate the left child, then the right child, then apply the node's operation. This explicit order defines which failure is observed first when both sides fail. The reference uses 64-bit intermediates for values bounded to one billion: multiplying two valid operands stays within the signed 64-bit range before checking the tighter language range.

A failed operation reports the operator's source offset. The interpreter returns one value only after the entire tree succeeds. It never partially updates the simulation during expression evaluation. Host changes happen only after the caller receives and validates a result.

Differential tests later compare tree evaluation with bytecode execution, but both share the declared arithmetic policy. Add independently calculated golden results and domain-error examples so a shared arithmetic mistake is not hidden by agreement between the two engines.

Parse and evaluate `2 + 3 * 4`

Tokens are Number(2), Plus, Number(3), Star, Number(4), End. The term parser groups 3*4 before the outer expression creates an addition node. The AST is Add(Literal(2), Mul(Literal(3),Literal(4))). Evaluation computes 3*4 = 12, then 2+12 = 14.

For `8 - 3 - 1`, the expression loop creates Sub(Sub(8,3),1), yielding 4. Right association would yield 6 and violate the specification. For `8 / 3`, the result is 2; for `-8 / 3`, it is -2.

For `1 / (2 - 2)`, syntax is valid but evaluation fails at the slash. For `2 3`, parsing fails because an extra Number remains before End. For `speed + 1`, the lexer reports the unsupported name rather than invoking a host variable.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_15_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
expression := term { ("+" | "-") term }
term       := primary { ("*" | "/") primary }
primary    := integer | "-" primary | "(" expression ")"
integer    := digit { digit }
digit      := "0" | "1" | ... | "9"
```

Common failure modes

Checking only nesting depth misses a long left-deep AST. Ignoring trailing tokens accepts malformed programs. A parser must not execute host text. An unknown name is not automatically zero. Shared evaluator agreement is not an independent arithmetic proof.

Practical assignment

Original requirement: Implement tokens, recursive-descent parsing and evaluation for numbers, parentheses and four arithmetic operators. Pass: test precedence, associativity, malformed input, unknown names and division-by-zero policy.

1. Write the grammar, numeric domain, evaluation order and error-location policy before coding.
2. Implement a bounded lexer retaining offsets and a recursive-descent parser producing an arena AST.
3. Implement evaluation with intermediate-range and division checks.
4. Test precedence, associativity, malformed input, unknown names and all declared resource limits.

Required cases: 2+3*4 -> *14; (2+3)*4 -> 20; 8-3-1 -> 4; -8/3 -> -2; 1/(2-2) -> division error at slash; empty/2 3/unknown name -> diagnostic; excess source, tokens, nodes or depth rejected.

Evidence to submit: Language specification, token/AST traces, golden-result table, located diagnostics and bounded-input tests.

Independent transfer: Add a remainder operator. Specify precedence, negative-operand behavior and zero-divisor policy before changing the lexer or parser.

Assigned primary reading: R14: Kaleidoscope lexer, parser and abstract syntax tree chapters.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What separates lexing from parsing?
2. Under ordinary precedence, what is 2+3*4?
3. Why preserve source locations in a syntax tree?
4. Why must the parser require End after the expression?
5. Why can an arena of indexed nodes be easier to own than pointers into a growable node vector?

15.3 — Bytecode, virtual machines and bounded execution

Learning outcome: Implement a small execution model with explicit limits.

A virtual machine is a specified execution model

The reference instruction set contains PUSH literal, ADD, SUB, MUL, DIV and NEG. PUSH adds one validated value to the stack. A binary arithmetic instruction consumes two operands and pushes one result; NEG consumes one and pushes one. The program has no jumps, host calls, files or network access.

Compile the AST in postorder: emit the left subtree, then the right subtree, then the operator. This order matches the tree evaluator's left-before-right error semantics. Instructions retain the source offset of the corresponding operation so runtime failures point back to the expression.

The code is an in-memory teaching representation, not a portable bytecode file format. An external byte format would need specified encoding, versioning, length validation and a decoder. Do not serialize the raw instruction structure and call it a cross-platform format. [R14: code generation concepts]

Operand order is easy to reverse accidentally

For binary subtraction, pop the right operand first and the left operand second, then compute left minus right. The top of stack contains the most recently compiled operand, not necessarily the mathematical left side. Addition can hide a reversal because it is commutative on ordinary valid integers; subtraction and division expose it.

Before popping, require at least two values. Before PUSH, require available stack capacity and a valid literal. Reject an unknown opcode through an explicit default case. At program end, require exactly one stack value. Empty code or two uncombined literals are invalid completed expressions under this VM contract.

The reference's stack has a configured maximum of 256 values. The compiler-produced expressions also obey the AST bounds, but the VM must not trust arbitrary instruction sequences merely because the normal compiler emits valid ones.

Bound every stage, not only instruction count

The VM accepts an execution budget and decrements it before each instruction. A nonempty program with budget zero fails before executing its first instruction. The reference also caps program length and stack size, and validates arithmetic at every operation.

Instruction fuel limits executed steps, not arbitrary wall-clock time. The current straight-line instructions are deliberately small and contain no unbounded host work. Adding a host callback, allocation-heavy instruction or loop changes the cost model and requires renewed limits. Parsing, lexing and AST construction already have their own limits because VM fuel cannot protect work performed before execution.

A bounded interpreter is not automatically a secure sandbox. Native implementation defects, unsafe host capabilities and environment exposure remain concerns. Keep this classroom VM local and capability-free. Production mod execution requires a separate security architecture and review. [P4-03; P4-04]

Compare engines without overstating independence

For every valid expression, compile it and compare VM output with tree evaluation. Check known results such as 2+3*4 = 14 independently. For failures, compare error category and source location where the language promises equivalence.

The reference tree evaluator and VM intentionally share arithmetic validation to avoid divergent definitions of division and range checks. Consequently their agreement is strong evidence about tree traversal, emission order and stack behavior, but weaker evidence about that shared arithmetic function. Golden examples and boundary tests address the shared dependency.

Add malformed bytecode tests that cannot come from the normal compiler: invalid opcode, underflow, out-of-range literal, excess stack, extra final values and insufficient fuel. Testing only compiler-generated bytecode leaves a large part of the VM's input contract unexplored.

Execute `8 - 3 * 2`

Bytecode is PUSH 8, PUSH 3, PUSH 2, MUL, SUB. Stack traces are [8], [8,3], [8,3,2], [8,6], [2]. MUL changes stack height by -1, as does SUB. For SUB, right = 6 and left = 8; reversing them would incorrectly produce -2.

The five-instruction program succeeds with budget 5 and fails with budget 4 before the final SUB. An isolated ADD fails before reading nonexistent operands. PUSH 1 followed by PUSH 2 fails at completion because two values remain.

Preserve the slash offset for a division instruction so `1/(2-2)` reports the same operation location as the tree evaluator.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_15_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
if (stack.size() < 2)
    throw LangError(instruction.pos, "stack underflow");
const auto right = stack.back(); stack.pop_back();
const auto left  = stack.back(); stack.pop_back();
stack.push_back(apply(instruction.op, left, right,
                      instruction.pos));
```

Common failure modes

Subtraction and division reveal reversed pops. Instruction fuel does not bound frontend work or arbitrary host callbacks. A bytecode vector is not a secure serialized format. Shared arithmetic means two evaluators are not completely independent oracles.

Practical assignment

Original requirement: Compile the expression language into stack bytecode and compare results to the tree evaluator. Pass: reject stack underflow, invalid opcodes and over-budget execution; preserve source context for errors.

1. Specify each opcode, stack effect, operand order and error location.
2. Compile the AST in postorder and compare golden stack traces.
3. Implement VM checks for malformed instructions and resource limits.
4. Differentially test valid expressions and error behavior, explicitly documenting shared validation code.

Required cases: 8-3*2 -> 2; five instructions pass with fuel 5, fail with 4; ADD on empty -> underflow; unknown opcode rejected; two final values rejected; PUSH beyond stack cap rejected; invalid literal and arithmetic range rejected.

Evidence to submit: Opcode table, stack traces, compiler/VM differential tests, malformed-bytecode tests and limit matrix.

Independent transfer: Add a conditional jump. Identify the new obligations for instruction-pointer validation, fuel, reachable stack states and source-level termination claims.

Assigned primary reading: R14: code generation and execution concepts; course security requirements.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What is the stack-height change of a binary add that pushes one result?
2. Why compare VM output to the tree evaluator?
3. Does using bytecode automatically create a secure sandbox?
4. Why do subtraction tests matter even when addition tests pass?
5. What does comparing two engines that share arithmetic validation fail to establish independently?

15.4 — Automata, computability and compiler optimization

Learning outcome: Know what analysis can establish and when transformation is legal.

Finite automata recognize bounded-memory patterns

A finite-state machine has a finite set of states, an initial state, transition rules and accepting states. A decimal-literal recognizer can use Start, Digits and Dead. From Start a digit enters Digits; from Digits another digit stays there; any other character enters Dead. Accept only if the final state is Digits.

The empty string is rejected because no digit was read. The recognizer accepts leading zeros under the stated grammar. It recognizes the digit pattern but does not by itself check the numeric value limit; an enormous decimal string can be lexically valid and numerically out of range.

A lexer uses related state transitions to classify tokens, but the complete expression grammar also has arbitrarily nested parentheses in its mathematical form. The teaching implementation imposes a depth limit. Keep lexical recognition, grammatical structure and implementation bounds distinct. [R06: computation models; R14: frontend]

Undecidability does not prohibit useful proofs

The halting problem says there is no general algorithm deciding termination for every program in a sufficiently expressive unrestricted computation model. It does not say that no particular program can be proven to terminate or that useful restricted analyses are impossible.

Our accepted expression AST is finite and acyclic. The tree evaluator visits a bounded number of nodes, and the compiled bytecode is straight-line with no jumps. Each operation is bounded by the chosen native integer implementation. These structural restrictions let us argue termination for accepted programs without solving a general halting problem.

A parser can still fail to terminate because of an implementation bug, such as a loop that never advances the token position. Prove progress of each loop and a decreasing measure or explicit bound for each recursion. A fuel counter is a runtime limit, not a proof that the source language's unrestricted future extensions always terminate.

Optimization must preserve the chosen observations

Constant folding replaces an operation on constant values with its result when doing so preserves the language's semantics. For this language, semantics include bounded integer arithmetic, left-before-right evaluation, error category and source offset. Folding 2+3 into 5 is valid; silently folding a range overflow into a wrapped number is not.

The reference optimizer attempts successful constant computations but leaves failing operations in place so their runtime errors remain observable at the same operator. It does not claim that every theoretically possible optimization is implemented. Since the baseline has no variables, successful whole expressions can often fold to one literal, making it a clear teaching demonstration rather than a realistic optimizing compiler.

Algebraic rewrites require stronger reasoning. Replacing `(1/0)*0` by 0 would remove a required error. In a floating-point language, reassociating `(a+b)+c` as `a+(b+c)` can change rounding. An optimization is legal only under the specified domain and observations, not because the algebra is familiar over real numbers. [R14: optimizer support]

Use equivalence tests, but retain the proof obligation

Compare original tree evaluation, unoptimized VM and optimized VM on independently specified examples and generated bounded expressions. For valid expressions require equal values. For failures require the promised error category and source offset. A set of tests supports confidence but is not a proof over all accepted programs.

Explain the transformation locally: children are folded only into semantically equivalent subtrees; a parent is replaced only when its constant computation succeeds under the same arithmetic rule; otherwise its operation and location remain. That inductive argument is more informative than saying a thousand tests passed.

Gate C integrates the native simulation, resource manager, data structures, pathfinding or tooling and expression language. You must be able to explain the complete source-to-execution path and modify one new grammar case without copying the reference. The required C-language bridge follows this module for the hardware direction.

A fold that preserves an error

For `2+3*4`, fold 3*4 to 12 and then 2+12 to 14. Compilation can emit one PUSH 14 rather than five stack instructions. The original and optimized paths both produce 14 under the defined range.

For `(1/0)*0`, division fails. The optimizer leaves the division node rather than treating the entire product as zero. Execution still reports the division error at the slash, and the multiplication result is never produced.

For a finite-state literal recognizer, trace `007`: Start -> Digits -> Digits -> Digits, accept. Trace `7x`: Start -> Digits -> Dead, reject. Trace empty input: remain Start, reject. Numeric conversion is a separate subsequent check.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_15_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// Conceptual folding rule:
fold children while preserving evaluation order;
if both operands are constant:
    try the language's checked arithmetic;
    if it succeeds, replace with a literal;
    if it fails, retain the operation and source location;
otherwise retain the operation;
```

Common failure modes

A lexical digit recognizer does not enforce numeric range. Undecidability is not a ban on proving a restricted evaluator terminates. A rewrite that removes a required error changes behavior. Passing equivalence tests is not a universal proof.

Practical assignment

Original requirement: Add constant folding to the toy compiler and test it against the unoptimized evaluator. Pass: include a finite-state recognizer, explain a termination restriction and document transformations deliberately excluded for semantic reasons.

1. Implement the finite-state decimal recognizer and separate lexical validity from numeric validity.
2. Add constant folding with explicit preservation of values and located runtime failures.
3. Compare original, bytecode and folded execution on golden and generated inputs.
4. Write a termination argument for the restricted language and list excluded transformations with semantic reasons.

Required cases: empty digits rejected; 007 accepted; 7x rejected; 2+3*4 folds to 14; (1/0)*0* retains division error; range overflow preserved as error; left-before-right error location retained; optimized and unoptimized valid results agree.

Evidence to submit: State-transition table, optimizer rules, equivalence tests, preserved-failure examples and restricted-termination argument.

Independent transfer: A proposed rewrite replaces `x-x` with zero after the language adds effectful function calls. Explain which assumptions would be required before approving it.

Assigned primary reading: R14: optimizer support; R06: computation models and proof limitations.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does undecidability mean no individual program can be proven to terminate?
2. Why can floating-point reassociation change a result?
3. What is the primary correctness condition for an optimization?
4. Why is replacing `(1/0)*0` by 0 invalid in this language?
5. What structural facts let this tiny evaluator terminate without a general halting solver?

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.